

POLITECNICO MILANO 1863 | SCUOLA DI INGEGNERIA INDUSTRIALE E DELL'INFORMAZIONE

Master's Degree in Computer Science and Engineering

Project report

"Requirements Analysis and Specification Document"

"Best Bike Paths"

Academic Year: 2025/2026

Authors: **Daniele Toniolo** **([redacted])**
Stefano Ungaro **([redacted])**
Matteo Zappa **([redacted])**

Professor: **Matteo Camilli**

Contents

Table of Contents **II**

List of Figures **IV**

List of Tables **V**

1 Introduction **1**
1.1 Purpose . . . 1
1.1.1 Goals . . . 1
1.2 Scope . . . 1
1.2.1 World Phenomena . . . 2
1.2.2 Shared Phenomena . . . 2
1.3 Definitions, Acronyms, Abbreviations . . . 3
1.3.1 Definitions . . . 3
1.3.2 Acronyms . . . 3
1.3.3 Abbreviations . . . 3
1.4 Revision History . . . 4
1.5 Reference Documents . . . 4
1.6 Document Structure . . . 4

2 Overall Description **5**
2.1 Product perspective . . . 5
2.1.1 Scenarios . . . 5
2.1.2 Domain Class Diagram . . . 6
2.1.3 State Diagrams . . . 8
2.2 Product functions . . . 10
2.2.1 Requirements . . . 10
2.3 User characteristics . . . 12
2.4 Assumptions, dependencies and constraints . . . 12
2.4.1 Regulatory policies . . . 12
2.4.2 Domain Assumptions . . . 13

3 Specific Requirements **14**
3.1 External Interface Requirements . . . 14
3.1.1 User Interfaces . . . 14
3.1.2 Hardware Interfaces . . . 16
3.1.3 Software Interfaces . . . 17
3.1.4 Communication Interfaces . . . 17
3.2 Functional Requirements . . . 18
3.2.1 Use Case Diagram . . . 18
3.2.2 Use Cases and Activity Diagrams . . . 19
3.2.3 Requirement Mapping . . . 39
3.3 Performance Requirements . . . 39
3.4 Design Requirements . . . 40
3.4.1 Standards compliance . . . 40
3.4.2 Hardware limitations . . . 40
3.4.3 Any other constraint . . . 41
3.5 Software System Attributes . . . 41
3.5.1 Reliability . . . 41
3.5.2 Availability . . . 41
3.5.3 Security . . . 42
3.5.4 Maintainability . . . 42
3.5.5 Portability . . . 42

4 Formal Analysis Using Alloy **43**
4.1 The Model Structure . . . 43
4.1.1 Signatures . . . 43
4.1.2 Predicates . . . 44
4.1.3 Facts . . . 45
4.1.4 Assertions . . . 47

4.1.5 Main Execution 48
4.2 Validation through the Model 50
4.2.1 Large Static World 50
4.2.2 Dynamic World 50
4.2.3 Assertion Checks 55

5 Effort Spent **56**

6 References **57**
6.1 Used Tools 57

List of Figures

2.1 Domain Class Diagram for BBP . . . 6
2.2 State Diagrams for Core Entities . . . 9

3.1 User Interfaces Map . . . 14
3.2 Use Case Diagram . . . 18
3.3 UC1 Activity Diagram . . . 20
3.4 UC2 Activity Diagram . . . 22
3.5 UC3 Activity Diagram . . . 24
3.6 UC4 Activity Diagram . . . 26
3.7 UC5 Activity Diagram . . . 28
3.8 UC6 Activity Diagram . . . 30
3.9 UC7 Activity Diagram . . . 32
3.10 UC8 Activity Diagram . . . 34
3.11 UC9 Activity Diagram . . . 36
3.12 UC10 Activity Diagram . . . 38

4.1 Large Static World Instance . . . 50
4.2 Dynamic World, Step 1: System Initialization and Sign Up . . . 50
4.3 Dynamic World, Step 2: Trip Recording . . . 51
4.4 Dynamic World, Step 3: User 1 Sign Up . . . 51
4.5 Dynamic World, Step 4: Manual Path Creation . . . 52
4.6 Dynamic World, Step 5: Path Publication . . . 52
4.7 Dynamic World, Step 6: User 3 Sign Up . . . 53
4.8 Dynamic World, Step 7: User 3 records a Trip . . . 53
4.9 Dynamic World, Step 8: Second Publication . . . 54
4.10 Dynamic World, Step 9: Third Publication . . . 54

List of Tables

1.1 Acronyms used in this document . . . 3
1.2 Abbreviations used in this document . . . 3
1.3 Revision history of this document . . . 4

2.1 User characteristics . . . 12

3.1 UC1: Sign Up by a User . . . 19
3.2 UC2: Log In by a User . . . 21
3.3 UC3: Access as Guest . . . 23
3.4 UC4: Record a Trip . . . 25
3.5 UC5: Validate a Trip and Path . . . 27
3.6 UC6: Change of Path's visibility . . . 29
3.7 UC7: Create a Path manually . . . 31
3.8 UC8: Search for a Path . . . 33
3.9 UC9: Delete a Trip . . . 35
3.10 UC10: Delete a Path . . . 37
3.11 Requirements Mapping . . . 39

5.1 Effort spent by each team member (unit is hours) . . . 56

Chapter 1

Introduction

1.1 Purpose

Cycling enthusiasts and daily commuters often face significant challenges in identifying safe and reliable routes for their journeys, as standard navigation tools frequently lack granular information regarding the physical condition of roads.

Best Bike Paths (BBP) is designed to fill this problem. The Platform aims to assist cyclists in their activities by creating a dynamic, crowdsourced inventory of bike Paths while enabling the community to effectively share data about Path statuses and obstacles. By addressing the mismatch between static map data and the actual road conditions, BBP enhances the routing process, creating an ecoSystem where User participation directly translates into safer and more informed cycling experiences.

1.1.1 Goals

[G1] Cyclists should be assisted in their cycling activities through the analysis of their Trips.

[G2] Cyclists should be able to record their Trips.

[G3] Cyclists should be able to view their Trips' analysis based on performance (e.g., speed, time, total distance) and on Paths (e.g., status, presence of obstacles, weather).

[G4] Cyclists should be able to provide information on Paths statuses manually.

[G5] Cyclists should have their Trips' Paths automatically recorded while cycling.

[G6] Cyclists should be able to access reliable and consolidated information about Paths (e.g., status, score) derived from the aggregation of community data.

[G7] Cyclists should be able to obtain a list of possible Paths from a source point to a destination point, along with their scores and statuses.

[G8] Cyclists should be able to suggest Paths to other Users.

1.2 Scope

Best Bike Paths (BBP) is a collaborative Platform designed to enhance the cycling experience by facilitating the collection and retrieval of granular road information. It serves a dual purpose: enabling Registered Users to track their performance and actively contribute to the community, while allowing Guest Users to safely navigate and explore the existing map inventory.

At the core of the System lies the Data Aggregation logic. By collecting heterogeneous User reports on road Segments (e.g., *Optimal*, *Needs Maintenance*), the Platform resolves conflicting information based on data freshness and confirmation counts, ultimately assigning a single, reliable quality score to each route.

To minimize User distraction and ensure data accuracy, the System automates key activities such as obstacle detection via accelerometer and gyroscope analysis, weather data retrieval, and performance statistics calculation. Furthermore, it provides dedicated tools for the manual validation of anomalies and the management of personal Trip history, ensuring a comprehensive and reliable service.

1

1.2.1 World Phenomena

[WP1] Users choose their preferred Path suggestion among a list of alternatives.

[WP2] Users start a cycling Trip.

[WP3] Users end a cycling Trip.

[WP4] During a ride, Users encounter obstacles along the way, such as potholes.

[WP5] The Users' mobile devices undergo physical vibrations and orientation changes while cycling.

[WP6] Paths conditions worsen during time.

[WP7] Municipalities or entities improve the physical state of Paths through maintenance or new constructions.

[WP8] Information regarding weather conditions is collected and made available by external meteorological services.

[WP9] Users decide to plan a Trip between two locations.

[WP10] Users decide to have their Trip recorded for analysis.

1.2.2 Shared Phenomena

World Controlled

[SP1] Users enter credentials to sign up to BBP.

[SP2] Users enter credentials to log in to BBP.

[SP3] Users enter the application without providing credentials (Guest access).

[SP4] Users manually provide details about Paths.

[SP5] Users begin a cycling session on BBP.

[SP6] Users end a cycling session on BBP.

[SP7] Users confirm the presence of an automatically detected obstacle.

[SP8] Users deny the presence of an automatically detected obstacle.

[SP9] Users delete a Trip.

[SP10] Users make their Trips' Path information public.

[SP11] Users insert a source point for Path research.

[SP12] Users insert a destination point for Path research.

[SP13] The User's mobile device transmits GPS coordinates to the System.

[SP14] The User's mobile device transmits accelerometer and gyroscope data to the System.

[SP15] The External Meteorological Service transmits weather data to the System.

Machine Controlled

[SP16] The System displays automatically detected obstacles to the User for confirmation.

[SP17] The System provides the Users with possible Paths from a source to a destination, including their scores.

[SP18] The System displays performance statistics (e.g., speed, distance) to the User.

[SP19] The System requests weather updates for a specific location to the External Meteorological Service.

1.3 Definitions, Acronyms, Abbreviations

1.3.1 Definitions

System, Platform These terms are used interchangeably in this document to refer to the Best Bike Paths (BBP) software and infrastructure.

Trip A continuous cycling activity recorded by a specific User, characterized by a temporal sequence of GPS coordinates and associated sensor data.

Segment The atomic unit of a Path (e.g., a stretch of road between two intersections or a trail section).

Path A logical sequence of geographical Segments connecting a starting point to a destination, which can be traversed by a bicycle. It is the static representation of a route, distinct from the dynamic *Trip* which is the act of traversing it.

Obstacle A physical anomaly detected along a Path that represents a non-optimal condition for riding. Examples include, but are not limited to, potholes, steep slopes, speed bumps, and muddy terrain.

Status A qualitative descriptor assigned to a Path or Segment (e.g., *Optimal*, *Sufficient*, *Requires Maintenance*) derived from User reports and aggregated community data.

Score A calculated numerical value representing the overall quality and efficiency of a Path. It is computed by the System based on the aggregated Status and other factors (e.g., obstacles) to rank search results.

Guest An unregistered User who interacts with the Platform solely for information retrieval (searching Paths) without the ability to contribute data or record Trips.

1.3.2 Acronyms

Acronym	Definition
BBP	Best Bike Paths
RASD	Requirements Analysis and Specification Document
API	Application Programming Interface
GPS	Global Positioning System
OTP	One Time Password
HTTPS	HyperText Transfer Protocol over Secure Socket Layer
JSON	JavaScript Object Notation
TLS	Transport Layer Security

Table 1.1 – Acronyms used in this document

1.3.3 Abbreviations

Abbreviation	Definition
G	Goal
WP	World Phenomenon
SP	Shared Phenomenon
DA	Domain Assumption
R	Requirement
UC	Use Case

Table 1.2 – Abbreviations used in this document

1

1.4 Revision History

Revised on	Version	Description
23/12/2025	1.0	Initial release of the document

Table 1.3 – Revision history of this document

1.5 Reference Documents

The information contained in this document is based on the following sources:

1. Di Nitto, Rossi, Camilli, *"A.Y. 2025-2026 Software Engineering 2 Requirement Engineering and Design Project"*, 2025.
2. ISO/IEC/IEEE 29148:2018 - ISO/IEC/IEEE International Standard, *"Systems and software engineering - Life cycle processes - Requirements engineering"*, 2018.

1.6 Document Structure

1. Introduction a brief description of the project derived from the analysis of the provided specifications; it highlights the product's goals and scope.

2. Overall Description a more detailed description of the product that includes real-world scenarios and analyses the System from a technical perspective.

3. Specific Requirements a detailed list of all requirements necessary to satisfy the project goals, and their relationship with domain assumptions.

4. Formal analysis using Alloy a formal description of a subset of real-world phenomena, analysed using Alloy to highlight inconsistencies between assumptions and the previously modelled phenomena.

5. Effort spent the time spent by each group member contributing to the creation of this document.

6. References references to documents or tools used in the preparation of this document.

Chapter 2

Overall Description

2.1 Product perspective

2.1.1 Scenarios

Scenario 1: Automated Recording and Sensor-based Detection

Daniele, an enthusiast cyclist, wants to track his daily commute to work. He opens the BBP app, logs in, and selects *Start Cycling*. As he rides, the System automatically records his GPS Path. Suddenly, he hits a deep pothole; his smartphone vibrates violently. The System's algorithm analyses the accelerometer and gyroscope data and flags a potential obstacle at that location.

Upon reaching her destination, Daniele selects *End Cycling*. The System presents a summary of the Trip, including performance statistics (speed, duration) and weather data (sunny, 22°C) retrieved from the external meteorological service. The System also asks him to validate the detected anomaly: *We detected a strong vibration. Was this a pothole?*. Daniele confirms the detection. Finally, he decides to make this Path data public to warn other cyclists.

Scenario 2: Trip Planning with Data Merging and Freshness Analysis

Stefano wants to plan a safe Sunday ride from his home to the city park. He navigates to the *Discover* section and enters the source and destination points. The System calculates possible routes based on the available public data.

One direct route exists, but the System marks it with a low score and a *Needs Maintenance* status. This happens because, although two Users rated it as *Optimal* last month, three new Users reported *Severe Potholes* yesterday. The System's merging algorithm prioritizes this fresher, confirming data. Consequently, BBP suggests an alternative, slightly longer route that has a high score and *Optimal* status confirmed by recent Trips. Stefano chooses the safer alternative.

Scenario 3: Manual Path Creation and Management

Matteo, an expert mountain biker, knows a shortcut through a forest trail that isn't mapped on standard road maps. He is currently at home and cannot ride it, so he logs into BBP and selects the *Create a Path* section.

Interacting with the map, he manually inputs the sequence of Segments connecting the entrance and exit of the trail. He assigns a specific status (*Difficult*, *Muddy*) to the route, which the System applies to all the involved Segments. Since he wants to immediately share this discovery with the community, he sets the visibility to *Public* and confirms the creation. The System saves the Path and immediately updates the global map aggregation.

Scenario 4: Registration and Profile Setup

Michele is a new User who wants to join the community. He downloads the app and selects *Sign Up*. He provides his email, name, and password. After receiving an OTP code via email and verifying his identity, his account is active. He then accesses his profile to set his preferences. He is now ready to record Trips or browse the existing inventory of bike Paths.

2

Scenario 5: Guest Exploration (The Tourist)

Tommaso is a tourist visiting the city for a weekend. He rents a bike and downloads BBP to find a safe route to the castle, but he does not want to create an account or share his personal details.

He selects *Guest Login* (or *Skip Registration*). The System allows him to access the *Discover* section and search for Paths. He finds a highly-rated route and follows it using the map. However, when he tries to press *Start Cycling* to record his performance, the System informs him that this feature requires a registered account. Tommaso declines the registration and continues to use the app solely for navigation and browsing public information.

Scenario 6: Smart Activity Detection and Trip Deletion

Daniele starts a new recording for a training session. During the ride, he encounters a red traffic light and stops; the BBP System detects the zero speed and automatically pauses the recording to preserve the accuracy of the performance statistics. Later, he decides to get off the bike and walk it across a pedestrian area to grab a coffee. The System analyzes the accelerometer patterns and GPS speed, correctly identifying the activity as *Walking* rather than *Cycling*, and keeps the recording paused, ensuring the pedestrian Segment is not added to the bike Path inventory.

When Daniele resumes cycling, the System automatically wakes up and continues the tracking. At the end of the day, reviewing his *My Trips* section, Daniele realizes this fragmented session is not useful for his long-term statistics. Since the System design ensures data integrity by disallowing manual modification of the recorded GPS track structure (to prevent data falsification), Daniele decides to discard the session entirely. He selects the Trip, taps *Delete*, and confirms the action, permanently removing the data from the System.

2.1.2 Domain Class Diagram

The Domain Class Diagram (Figure 2.1) illustrates the conceptual model of the BBP System, defining the entities, their attributes, and the relationships that govern the System's logic. The model is designed to support the core requirements of crowdsourced data collection, Path reliability scoring, and User privacy.

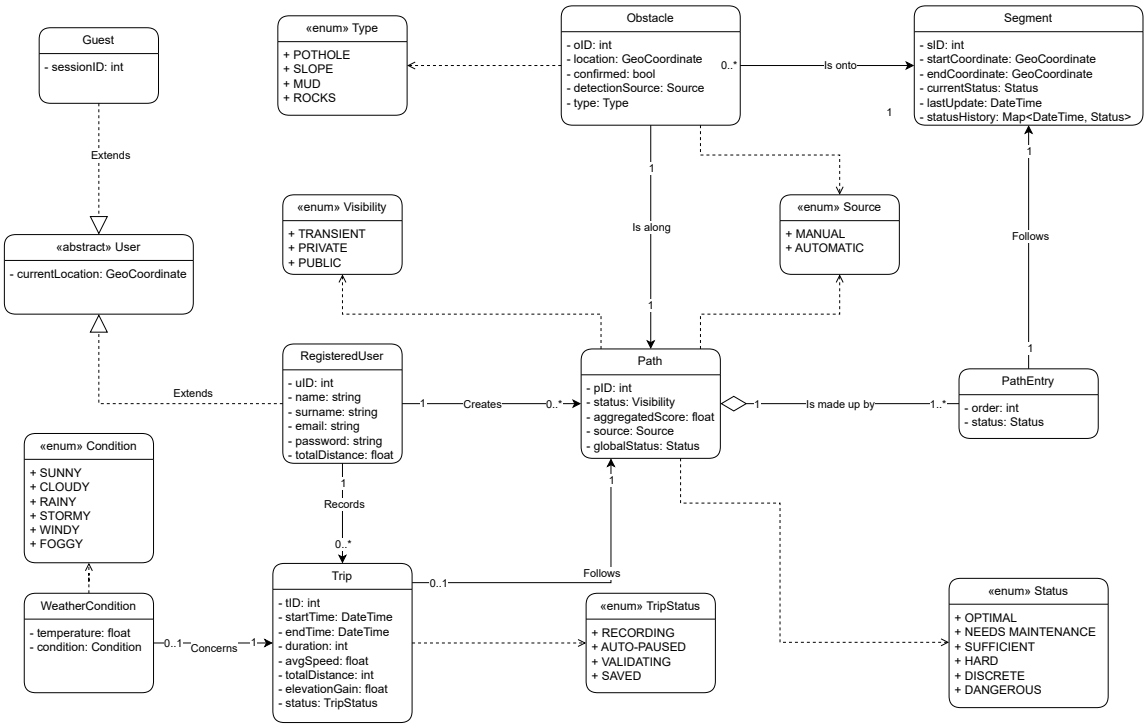


Figure 2.1 – Domain Class Diagram for BBP

The structure is centered around a clear separation between the *Dynamic Activity* (the act of cycling) and the *Static Geography* (the physical road network). This separation is crucial for the implementation of the Data Aggregation algorithm.

User Hierarchy

The System defines a generalization hierarchy to manage different levels of access:

- **User (Abstract)**: Represents the generalized concept of an actor interacting with the System via a mobile device. It holds the `currentLocation` (GeoCoordinate), which is the only attribute shared by all actors to enable map visualization and real-time tracking.
- **RegisteredUser**: Extends the generic User with persistent identity attributes (`email`, `password`), `name`, `surname`, and metrics such as `totalDistance`. This entity is the *owner* of data: only a RegisteredUser can record **Trips** and create **Paths**.
- **Guest**: Represents an anonymous User. This entity has no persistent ID or profile data, consistent with the privacy-by-design approach, but inherits the capability to search and view the map. A `sessionID` is needed for session persistence.

Core Entities: Trip, Path, and Segment

To satisfy the requirement of merging conflicting data from multiple Users while preserving the integrity of individual reports, the model adopts a normalized structure mediated by the **PathEntry** class:

- **Trip**: Represents the temporal event of a User cycling. It contains raw sensor data and performance metrics specific to that single session, such as `avgSpeed`, `totalDistance`, and `elevationGain`. Crucially, it includes a `status` attribute (of type `TripStatus`) to track the recording lifecycle (e.g., Recording, Auto-Paused, Saved). A Trip is transient in nature; it documents *what happened* at a specific time.
- **Path**: Represents the immutable geographical definition of a route. It acts as a container for metadata such as `aggregatedScore` and `globalStatus`. It holds a `status` attribute (of type `Visibility`) which manages the lifecycle and visibility of the route (e.g., Transient, Private, Public). The relationship between Trip and Path is defined as follows:
  - A Trip *follows* exactly one Path geometry (`1`).
  - A Path is usually generated by a Trip, but can also be created manually without any recording. Hence, the multiplicity is `0..1` on the Trip side.
- **PathEntry**: This entity acts as the structural link between a user-defined Path and the physical road network. A Path is composed of a sequence of `PathEntries` (ordered by the `order` attribute). Crucially, `PathEntry` holds the `status` assigned by the user during that specific trip. This ensures that the user's subjective rating is encapsulated within their private Path data and is only propagated to the global map when the Path becomes Public.
- **Segment**: This is the core entity for the **Data Aggregation** logic. A `Segment` represents a physical, atomic stretch of road (e.g., a specific street block or trail section) that exists independently of any User.
  - **Aggregation Logic**: The Segment holds the `currentStatus` (e.g., *Muddy*) and the `lastUpdate` timestamp, representing the "global truth" visible to all users.
  - **Status Propagation**: When a Path is published, the aggregation algorithm iterates through the list of `PathEntries`, taking the user-specific ratings and merging them into the `currentStatus` of the corresponding Segments. The Segment also maintains a `statusHistory` map to allow for data rollback and validation.

Context and Anomalies

The model includes entities to capture the environmental context of the ride:

- **Obstacle**: Represents a hazard (e.g., *Pothole*, *Slope*) detected along a route. Obstacles lie on specific Segments and are connected to a specific Path. Crucially, the Obstacle includes a `detectionSource` attribute (defined in the `Source` enumeration) to distinguish between anomalies automatically detected by the sensors (accelerometer/gyroscope) and those manually inserted by the User during validation.
- **WeatherCondition**: Associated with every Trip, when available, it freezes the meteorological conditions (Temperature, Condition) at the time of the recording. This allows the System to filter statistics (e.g., *Performance on rainy days*).

Enumerations

To ensure data consistency, the model relies on strict enumerations for categorical data:

- **Status**: Defines the quality of a Segment (Optimal, Sufficient, Hard, Dangerous).
- **Type**: Classifies the hazards (Pothole, Mud, Rocks).
- **Condition**: Standardizes weather reports (Sunny, Rainy, Foggy).
- **Source**: Distinguishes between Manual and Automatic obstacle detection.
- **TripStatus** and **Visibility**: Define the lifecycle states for the respective entities.

These enumerations can be extended with additional entries for a more detailed representation of different aspects of the object they refer to.

2.1.3 State Diagrams

To clarify the dynamic behaviour of the System's core entities, the following state diagrams illustrate the lifecycle of a **Trip** (focusing on the recording logic) and a **Path** (focusing on data persistence and visibility).

Trip Lifecycle

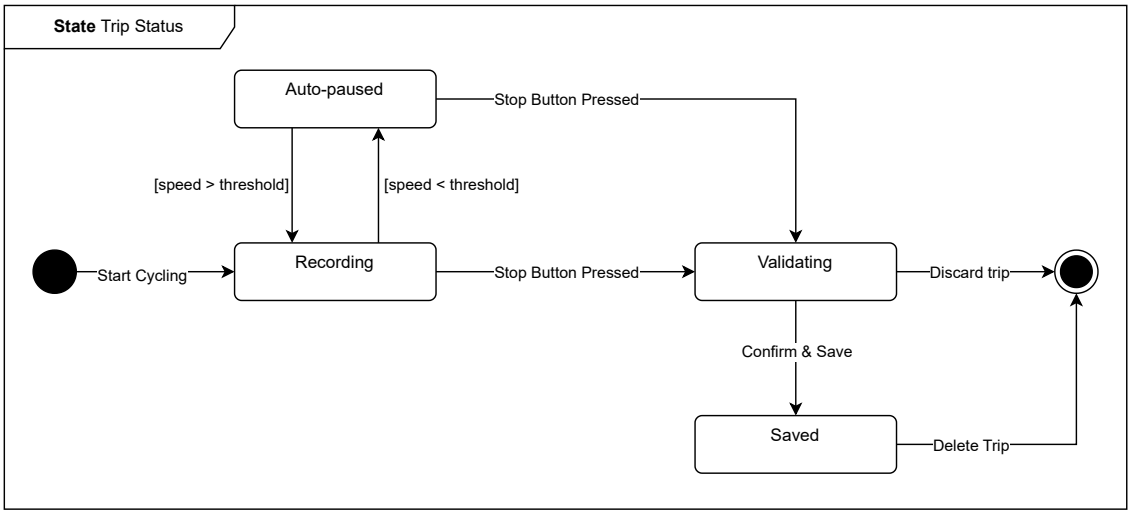
Figure 2.2a models the User's recording session. It highlights the *Smart Pausing* mechanism defined in requirement [R9], ensuring that stops or non-cycling activities do not pollute the performance statistics.

- **Recording**: The active state where the System logs GPS coordinates and sensor data.
- **Auto-Paused**: Entered automatically when the System detects the speed has dropped below a threshold (for example, if the User is walking). In this state, data logging is suspended. The System automatically reverts to *Recording* once cycling resumes.
- **Validating**: A transitional state entered upon pressing the *Stop* button. Here, the data is frozen while the User confirms details about the Path and assigns a status (as per UC5). From here, the User can either confirm the session or discard it entirely.
- **Saved**: The final stable state where the Trip is persisted in the User's history (*My Trips* section). It remains here until the User explicitly decides to delete it.

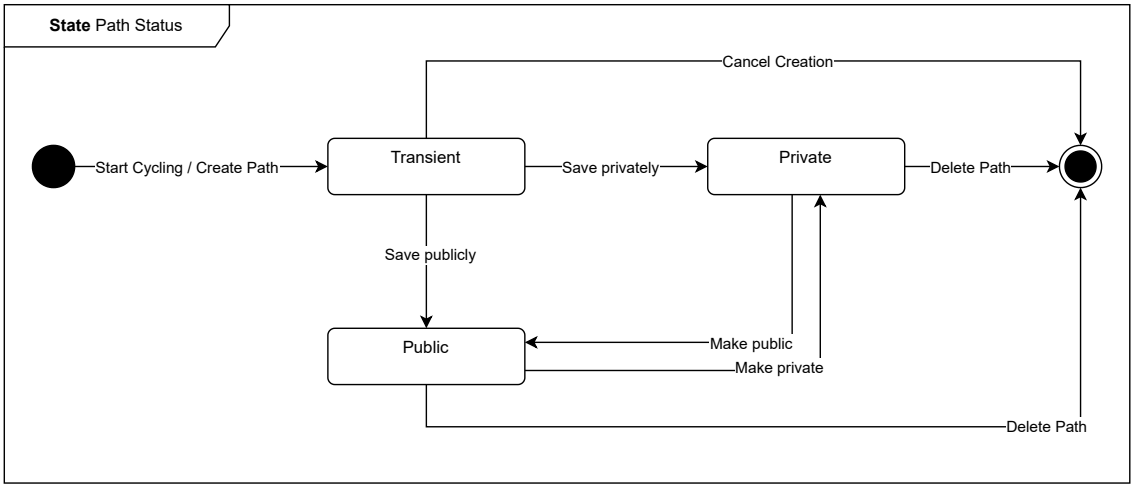
Path Lifecycle

Figure 2.2b illustrates the lifecycle of the geographical route definition, managing its visibility to the community. This logic is central to the *Data Merging* requirements.

- **Transient**: The Path is being defined (either by a recording session or manual drawing via UC7) but is not yet persisted in the database. If the creation is cancelled, the object is destroyed.
- **Private**: The Path is visible only to its creator and does not contribute to the global map statistics.
- **Public**: The User has explicitly shared the Path. In this state, the Path's Segment statuses are merged into the global heatmap using the aggregation algorithm. A Path can be toggled between *Private* and *Public* at any time, triggering a recalculation of the community data.



(a) State Diagram of a Trip



(b) State Diagram of a Path

Figure 2.2 – State Diagrams for Core Entities

2.2 Product functions

The BBP Platform is designed to help cyclists manage their Trips and Paths flawlessly, with key features that help make the User experience friendly and optimal for monitoring and planning.

2

- **User Access and Registration**: Users can sign up to the Platform to enable tracking features and contribute to the community data. The System also supports a *Guest Mode*, allowing unregistered Users to browse the inventory and search for Paths without recording their activities.
- **Trip Tracking and Statistics**: The Platform provides a tracking functionality that Users can activate when starting a new Trip. While cycling, the application uses the built-in phone sensor System (GPS, accelerometer, gyroscope) to record the session and analyse performance (e.g., speed, duration). If available, the System enriches the Trip data with meteorological information retrieved from external services.
- **Automated Data Acquisition**: During a Trip, the System automatically detects the Path structure and identifies potential anomalies (e.g., potholes) by analysing sensor data. It features activity recognition to distinguish whether the User is cycling based on speed and movement patterns.
- **Data Validation**: To mitigate false positives in automated detection, the System provides an interface for Users to review, confirm, or correct the automatically detected obstacles and Path Segments before saving or publishing the Trip.
- **Manual Path Management**: The System allows Users to define Paths manually without cycling, by selecting Segments on a map and assigning a status (e.g., *Optimal*, *Needs Maintenance*) which applies to the selected road sections. Users can manage the visibility of their Paths, choosing to keep them private or share them with the community.
- **Data Aggregation and Reliability Analysis**: The System processes public information incoming from different Users to create a consolidated view of the bike Paths network. It merges conflicting reports by considering data freshness and the number of confirming sources to assign a reliability score and a unified status to each Path.
- **Smart Path Search**: Users can search for Paths between two locations. The System visualizes the results on a map, filtering and ranking the alternatives based on the calculated score, Path status, and overall efficiency.

2.2.1 Requirements

Account and Session

[R1] Upon request, the System shall allow a User to sign up to the Platform, as long as they provide all the required information and a profile is not already associated with it.

[R2] Upon request, the System shall allow a User to log in to the Platform, as long as the correct credentials are provided.

[R3] Upon request, the System shall allow a User to access the Platform as a guest.

[R4] Upon request, the System shall allow a registered User to start a training session.

[R5] Upon request, the System shall allow a registered User to end a training session, as long as they had already started it before.

Recording and Sensors

[R6] Whenever a Trip is being recorded, the System shall keep track of the Path followed by the User acquiring GPS coordinates.

[R7] Whenever a Trip is being recorded, the System shall analyze data from the device's accelerometer and gyroscope to automatically detect potential obstacles (e.g., potholes).

[R8] Whenever the System detects a potential obstacle during a recording, it shall temporarily store its location and type for validation at the end of the session.

[R9] Whenever a Trip is being recorded, the System shall automatically pause the data acquisition if the detected speed or movement pattern suggests the User is not cycling (e.g., walking or stationary).

Validation and Immutability

[R10] Upon concluding a recorded Trip (and before final storage), the System shall require the User to validate (confirm or delete) the automatically detected obstacles and assign a general status to the traversed Segments.

[R11] Upon concluding a recorded Trip (and before final storage), the System shall allow the User to manually insert and classify obstacles that were not automatically detected by the sensors.

[R12] The System shall prevent any modification to the Path's structural data (GPS track) and descriptive details (status, obstacles) once the Path has been saved to the System.

Manual Management

[R13] Upon request, the System shall allow a registered User to insert a Path manually, as long as a list of streets and statuses is provided.

[R14] Upon request, the System shall allow a registered User to delete their recorded Trips.

[R15] Upon request, the System shall allow a registered User to delete their Paths, whether inserted manually or automatically.

Visibility

[R16] Upon request, the System shall allow a registered User to make their Path available to the community (public), as long as said Path is currently private.

[R17] Upon request, the System shall allow a registered User to make their Path private, as long as said Path has already been shared with the community.

[R18] Upon request, the System shall allow a registered User to view all their Trips and Paths, whether they are private or public.

Data Merging and Algorithms

[R19] Whenever a Path is made public, the System shall recalculate the status of the involved street Segments by aggregating the User's data with existing community data.

[R20] When aggregating conflicting data about a Path Segment, the System shall prioritize information based on its freshness (recency of submission).

[R21] When aggregating conflicting data about a Path Segment, the System shall resolve inconsistencies by considering the number of confirming reports from different Users.

[R22] Whenever a public Path is made private or deleted, the System shall remove the specific User's contribution from the aggregated community data, triggering a recalculation of the Segment's status.

Statistics and Information

[R23] Whenever a Trip is recorded, the System shall elaborate statistics on performance, such as estimated speed, total length, and time.

[R24] Whenever a Trip is concluded and external weather services are available, the System shall retrieve and store the weather conditions associated with the Trip's location and time.

Search

[R25] Upon request, the System shall provide a list of Paths connecting two locations to any User (registered or guest), visualizing them on a map ordered by their calculated score.

[R26] Whenever providing search results, the System shall calculate a Path's score based on its aggregated status (e.g., optimal vs. bad) and its effectiveness in reaching the destination.

2.3 User characteristics

The BBP Platform is designed for two distinct categories of Users, differentiated by their level of interaction with the System's database:

User	Characteristics
Registered Cyclists	**Description**: Individuals who actively use the Platform not only to find routes but also to record Trips, monitor performance, and contribute data to the community. They span from *proficient Users*, familiar with fitness apps and navigation technology, to *novice Users* who may be less experienced with such interfaces. **Interaction Frequency**: Expert cyclists typically access the Platform multiple times per week for training. Recreational Users show seasonal variability (higher activity in spring/autumn and weekends). **Key Role**: They are the primary source of information for the System (crowdsourcing). Therefore, their ability to easily confirm or correct automated data is crucial for the System's reliability.
Unregistered Users (Guests)	**Description**: Individuals who wish to access the public inventory of bike Paths for exploration without account creation or performance tracking. **Interaction Frequency**: Highly unpredictable. This category includes tourists using the app for a single weekend or casual Users evaluating the service before registering. **Key Role**: They act solely as information consumers and cannot contribute new data or modify existing Paths.

Table 2.1 – User characteristics

Given that the User base ranges from highly technical Users to complete beginners, and considering that data quality relies on User validation, the Platform must ensure a high usability standard. The interface must be straightforward enough for a casual User to navigate without training, yet comprehensive enough to provide detailed analytics for the expert cyclist.

2.4 Assumptions, dependencies and constraints

2.4.1 Regulatory policies

Since the BBP Platform collects and processes sensitive User data, specifically real-time geolocation (GPS) and movement patterns (via accelerometer and gyroscope), compliance with the *General Data Protection Regulation*

(GDPR) is mandatory. Specific policies include:

- **Data Anonymization**: Before being merged into the public knowledge base or used for statistical analysis, raw Trip data must be anonymized to prevent the identification of specific individuals or their private locations (e.g., home addresses).
- **Purpose Limitation**: Data is collected strictly for the purpose of service provision (Trip tracking, Path analysis) and must not be used for third-party commercial profiling or sold to advertisers.
- **Explicit Consent**: The System must obtain explicit User consent before accessing device sensors and location services.

2.4.2 Domain Assumptions

[DA1] Users' GPS location is accurate.

[DA2] Users interact with the BBP application through a device with a functioning GPS connection.

[DA3] Cyclists that decide to sign up have access to the mailbox of the provided email address.

[DA4] Users do not provide harmful information on Paths, such as suggesting cycling in dangerous areas or malicious routes.

[DA5] The majority of Users participate in the community with the intent of providing accurate information. Consequently, statistical aggregation of conflicting reports converges towards the actual state of the Paths.

[DA6] Cyclists follow local traffic laws (e.g. no access to highways, motorways, or forbidden/private streets).

[DA7] Cyclists travel along physically defined tracks (roads, bike lanes, or beaten trails) and do not traverse open terrain where no Path exists (e.g., cutting diagonally across a field).

[DA8] The external map provider's database used by the System includes all physical Paths suitable for cycling, including off-road trails that might be missing from standard street navigation maps.

[DA9] During a Trip, cyclists rely only on their bicycle and do not switch to other means of transport (e.g. cars, trains, buses) without stopping the recording.

[DA10] An Internet connection is available for data synchronization (Log In, Upload, Search), although it may be intermittent or absent during the cycling activity itself.

Chapter 3

Specific Requirements

3.1 External Interface Requirements

3.1.1 User Interfaces

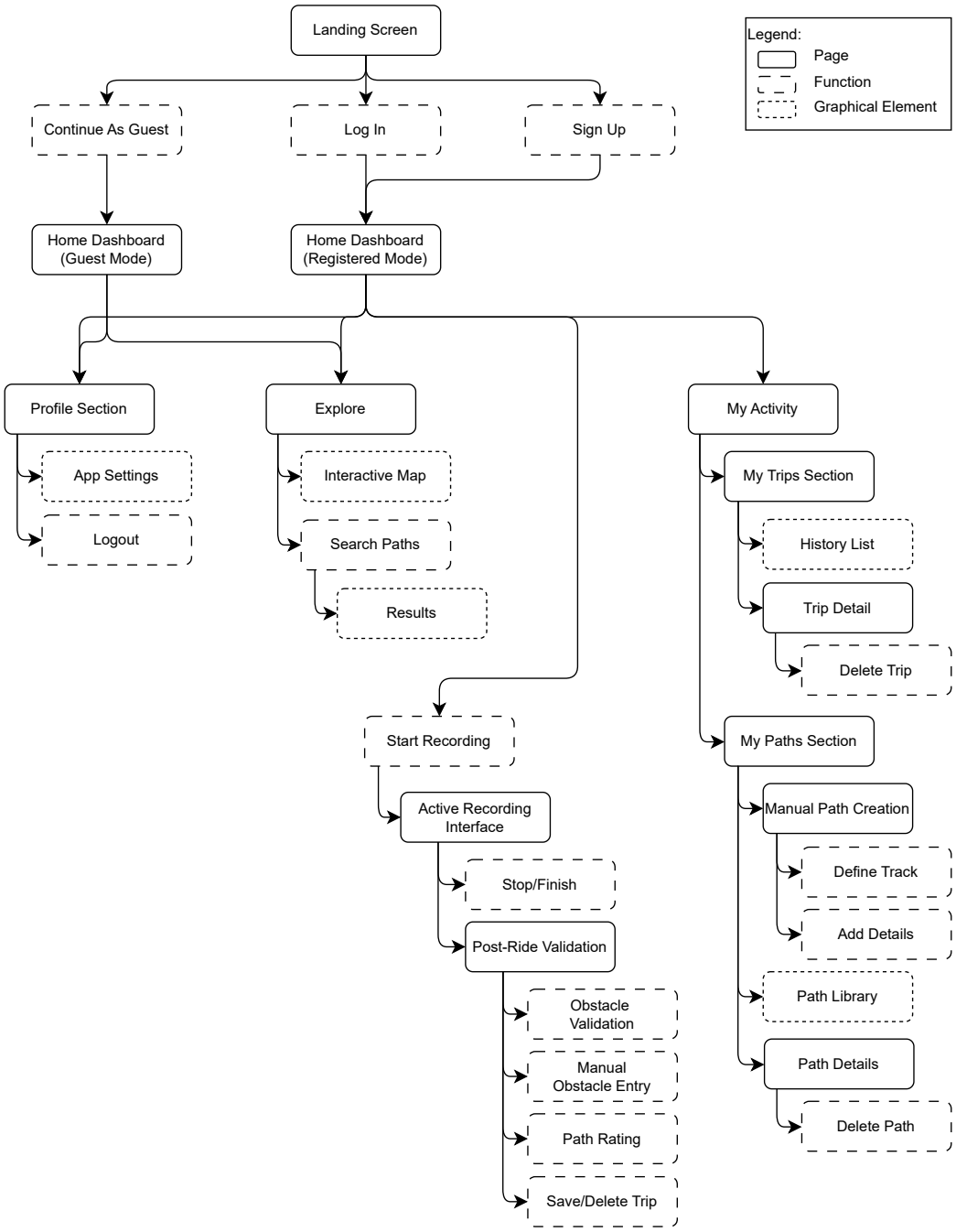


Figure 3.1 – User Interfaces Map

The Best Bike Paths (BBP) User interface shall be designed to be intuitive and strictly adherent to a *map-centric* approach, considering that the application will be used primarily on mobile devices in outdoor settings. The interactions shall be optimized for single-handed usage and high legibility while moving.

In compliance with accessibility requirements and store guidelines, the interface shall use high-contrast colour schemes, scalable fonts, and enlarged touch targets to facilitate interaction during physical activity.

The main System screens are described below:

- **Landing Screen (Authentication)**: Upon launching the app, the User is presented with a mandatory *Safety Disclaimer* regarding road risks. Once acknowledged, the screen offers clear entry points:
  - *Log In / Sign Up*: Standard input fields for registered Users to access their profile.
  - *Continue as Guest*: A prominent, high-visibility button allowing immediate access to map exploration without account creation.
- **Home Dashboard**: The central hub of the application, existing in two distinct versions: *Guest Mode* (with restricted features) and *Registered Mode* (providing full access).
  - *Interactive Map*: The core interface element occupying the majority of the screen, displaying the User's real-time position and nearby cycling Paths colour-coded by their aggregated status (e.g., Green for Optimal, Red for Damaged).
  - *Search Bar*: Floating at the top, allowing quick entry of source and destination points.
  - *Start Recording*: A large Floating Action Button to begin a Trip, available exclusively in Registered Mode.
  - *Bottom Navigation Bar*: Provides quick access to *Explore*, *Profile* and, for registered Users only, *My Activity* (with subsections for Trips/Paths).
- **Profile Section**: Manages global application preferences and User session state. The interface adapts its content based on whether the User is Registered or navigating as a Guest.
  - *App Settings*: Manages app-specific technical configurations and privacy controls. While sensor usage consent is available to all, specific identity management features (e.g., GDPR data export, Account Deletion) are hidden for Guest Users.
  - *Authentication Controls*: A dynamic area that manages the current session status.
    - For **Registered Users**, it displays a *Logout* button to terminate the session.
    - For **Guest Users**, it displays *Log In* and *Sign Up* buttons (or a banner) to encourage account creation and unlock full features.
- **Explore**: Serving as the application's main page for navigation, this section is centered around the *Interactive Map*, which allows Users to browse the cycling network directly. It also provides access to the search functionality, detailed as follows:
  - **Search Paths & Results**: Upon entering a source and destination, the interface presents *Path Alternatives* via a list or map overlay showing possible routes. Each option highlights the *Score* (calculated by the merging algorithm) and the aggregated *Status* , alongside *Warnings* represented by icons indicating specific issues reported by the community (e.g., *Muddy section*).
- **My Activity**: This section is divided into two primary subsections:
  - **My Trips Section**: Contains the chronological *History List* of activities and the *Trip Detail* view featuring graphs and maps, from which the *Delete Trip* action can be performed.
  - **My Paths Section**: This section manages the repository of saved Paths, featuring a *Path Library* that distinguishes between private and public entries. Users can access individual *Path Details* to view metadata or execute the *Delete Path* command to remove orphan Paths.
    - *Manual Path Creation*: Accessible here, it allows Users to define a Path without cycling via the *Define Track* map editor and the *Add Details* form for status and visibility.

- **Recording Flow**:The tracking process is managed through two primary interfaces designed to minimize distraction and ensure data quality:
  - **Active Recording Interface**: It is optimized for use while moving, displaying real-time metrics (speed, distance) and providing visual feedback for *Auto-Pause* events. To prevent accidental interruptions during the ride, the *Stop/Finish* function utilizes a deliberate "slide-to-confirm" interaction to terminate the session.
  - Upon completion, the System presents the **Post-Ride Validation** screen, a critical summary for data aggregation displayed immediately after finishing a recording. In addition to presenting a final snapshot of performance statistics (duration, average speed) and the weather conditions automatically retrieved for the Trip's location , this interface allows the User to toggle the Path visibility between *Private* and *Public* before finalizing the following data:
    - *Obstacle Validation*: A list allowing the User to confirm or reject anomalies automatically detected by the sensors.
    - *Manual Obstacle Entry*: A specialized interaction allowing the User to manually drop a marker on the map to categorize obstacles that the automated sensors failed to detect during the ride.
    - *Path Rating*: A tool to assign a general status to the traversed Path (e.g., *Optimal*). While this applies to all Segments by default, the User can override the status for specific map sections (e.g., marking a single stretch as *Muddy*).
    - *Save/Delete Trip*: The final step to confirm saving the Trip to the database or permanently deleting the recording.

3.1.2 Hardware Interfaces

The BBP mobile application shall interact directly with specific hardware components embedded in the User's mobile device to fulfil its core functional requirements (tracking and sensing). The interactions are managed through the abstraction layers provided by the mobile operating System:

- **Global Navigation Satellite System (GNSS) Receiver**: The application shall interface with the device's GPS/GNSS module to acquire real-time geospatial coordinates (latitude, longitude, altitude) and speed. The interface shall support background acquisition modes to allow Path recording while the screen is locked or the app is in the background.
- **Inertial Measurement Unit (IMU)**: To support the *Automated Data Acquisition* and obstacle detection features, the application shall interface with the motion sensors:
  - **Accelerometer**: To detect sudden vertical accelerations (shocks) indicative of potholes or irregular terrain.
  - **Gyroscope**: To measure device orientation and rotation, helping to distinguish between a bicycle turning and random device movement.
- **Network Interface Controller (Radio Modem)**: The application shall control the device's Wi-Fi and Cellular (4G/5G) signals to transmit data to the backend. The software shall detect the state of the hardware link to trigger the *Store-and-Forward* mechanism when the signal is lost.
- **Non-Volatile Storage (Flash Memory)**: The application interfaces with the internal storage hardware to persist the local database and map cache, ensuring data integrity during offline operations.
- **Hosting Infrastructure (Server-Side)**: The backend components shall interface with the underlying server infrastructure (typically provided by a Cloud Service Provider via IaaS/PaaS) to access the necessary computational resources:
  - **Processing Units (vCPU)**: To execute the heavy *Data Aggregation* and *Path Finding* algorithms. The infrastructure must support horizontal scaling (adding more nodes) to handle concurrent load peaks.
  - **High-Throughput Storage**: Fast I/O storage interfaces are required for the Database Management System to perform rapid geospatial queries and write operations for incoming Trip logs.

  - **Broadband Network Connection**: The server hardware must provide a high-bandwidth uplink to handle simultaneous data synchronization from thousands of mobile clients without bottlenecking.

3.1.3 Software Interfaces

The BBP System interacts with various software components to function correctly across client and server environments:

- **Mobile Operating Systems**: The client application shall be compatible with modern mobile operating Systems to maximize User reach. Specifically:
  - **Android**: Support for Android 10 (API level 29) and higher.
  - **iOS**: Support for iOS 15 and higher.

  The application interfaces directly with the OS SDKs to access low-level hardware services (Location Manager, Sensor Manager for accelerometer/gyroscope) and background processing tasks.
- **Database Management Systems (DBMS)**:
  - **Server-Side (Backend)**: The System shall utilize a DBMS with native support for geospatial queries to efficiently store and index Path coordinates, Segments, and spatial metadata.
  - **Client-Side (Local Cache)**: To support data caching and synchronization requirements, the mobile application shall interface with a local embedded database. This allows for the temporary persistence of Trip recordings and map tiles when network connectivity is unavailable.
- **External APIs**:
  - **Map Service Provider**: The System shall interface with an external mapping API to retrieve vector map tiles for visualization and graph data for routing algorithms.
  - **Meteorological Service**: The System shall interface with a third-party Weather API to retrieve historical and current weather conditions based on the timestamp and geolocation of the recorded Trips.
- **Development Framework Runtime**: Since the application is developed as a cross-Platform solution, it shall rely on the specific runtime environment of the chosen framework to translate application logic into native OS commands.

3.1.4 Communication Interfaces

The BBP System shall rely on standard network protocols to ensure secure and efficient data exchange between the mobile client, the backend server, and external third-party services:

- **Communication Protocols**:
  - **Client-Server**: The mobile application shall communicate with the BBP backend via a RESTful API over HTTPS. This ensures that all API endpoints (e.g., for login, Trip upload, Path search) are accessed using a standardized, stateless architecture.
  - **Server-External Services**: The backend shall interact with the Weather and Map providers using their respective public APIs, strictly over HTTPS to maintain the chain of trust and security.
- **Data Serialization**: Data exchanged between the client and the server (e.g., User profiles, Trip coordinates, sensor metadata) shall be serialized in JSON format. This choice minimizes data payload size, critical for mobile Users with limited data plans, and ensures compatibility with modern development frameworks.
- **Security in Transit**: To comply with GDPR and protect sensitive geolocation data from interception (Man-in-the-Middle attacks), all network traffic shall be encrypted using TLS 1.2 or higher. Unencrypted HTTP traffic is strictly prohibited.

- **Synchronization Mechanism**: Given the constraint of intermittent network coverage (as defined in Domain Assumptions), the communication interface shall implement an Asynchronous Store-and-Forward pattern. When the device is offline, the communication layer shall queue API requests (e.g., *Upload Trip*) in the local storage. A background *Connectivity Manager* shall monitor the network status and automatically flushes the queue once a stable internet connection is re-established, ensuring no data is lost due to connectivity drops.

3

3.2 Functional Requirements

3.2.1 Use Case Diagram

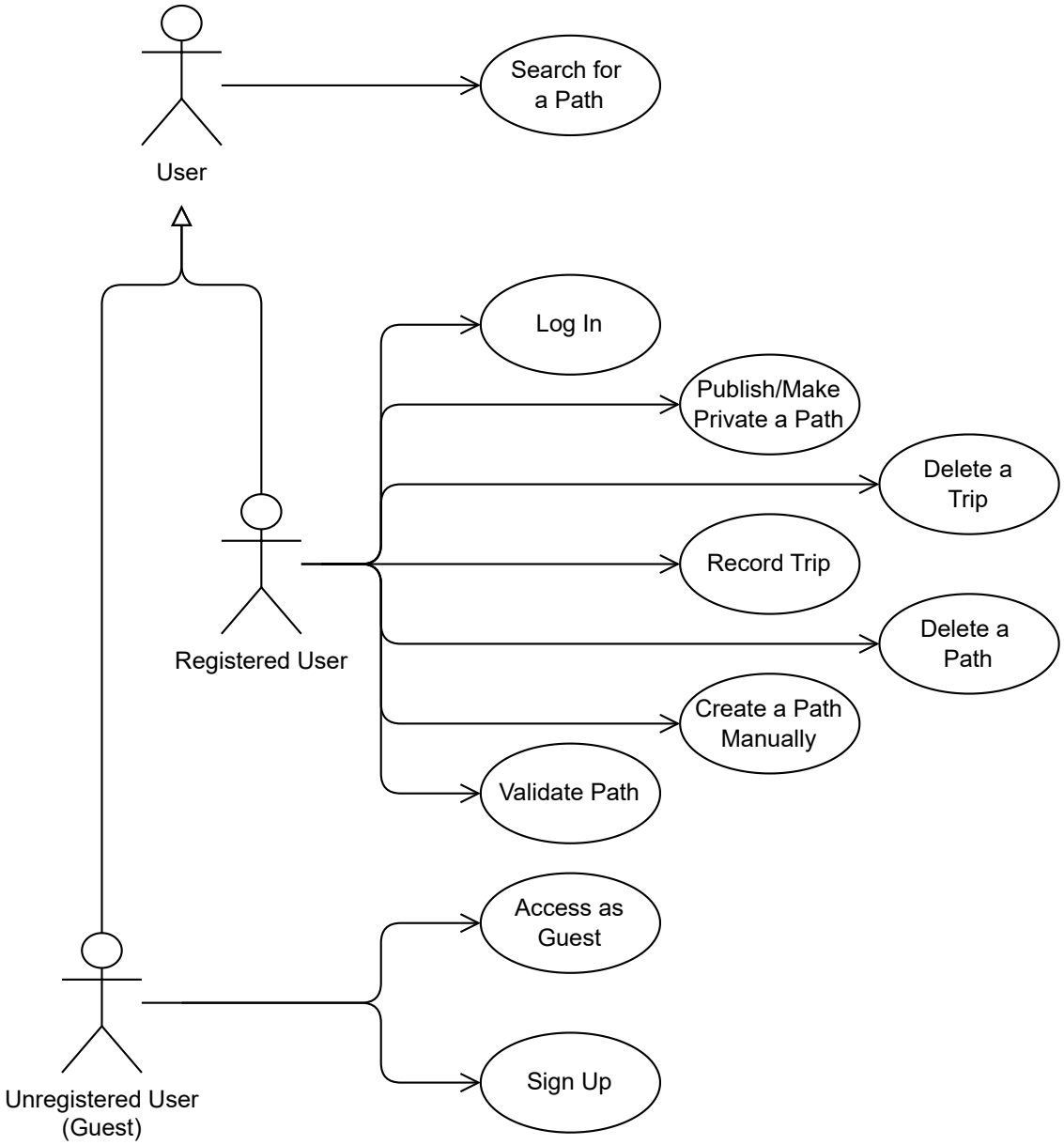


Figure 3.2 – Use Case Diagram

3.2.2 Use Cases and Activity Diagrams

[UC1] Sign Up by a User

Actor	Unregistered User (Guest)
Entry Conditions	The User is not logged into the software.
Flow of Events	1. On the homepage, the User clicks the *Sign Up* button, entering the initial registration page. 2. The User provides the required details: name, surname and his other personal details, email, password and password confirmation. 3. The User confirms the provided information by clicking the *Register* button. 4. The System verifies that the information is valid and that there is no existing profile associated with that email address. 5. The System sends an OTP (One Time Password) to the email address provided to verify identity. 6. The User enters the received OTP into the application. 7. The System validates the OTP and activates the User's account. 8. The User i redirected to the *Explore* page.
Exit Conditions	The profile is complete and the User has access to its functionalities.
Exceptions	• Existing profile: If the email address provided is already associated with an account, the System will deny registration and inform the User. • OTP verification failed: If the OTP entered is incorrect or does not match, the identity is not verified and the account is not activated until a correct one is provided (implicit in the verification process).

Table 3.1 – UC1: Sign Up by a User

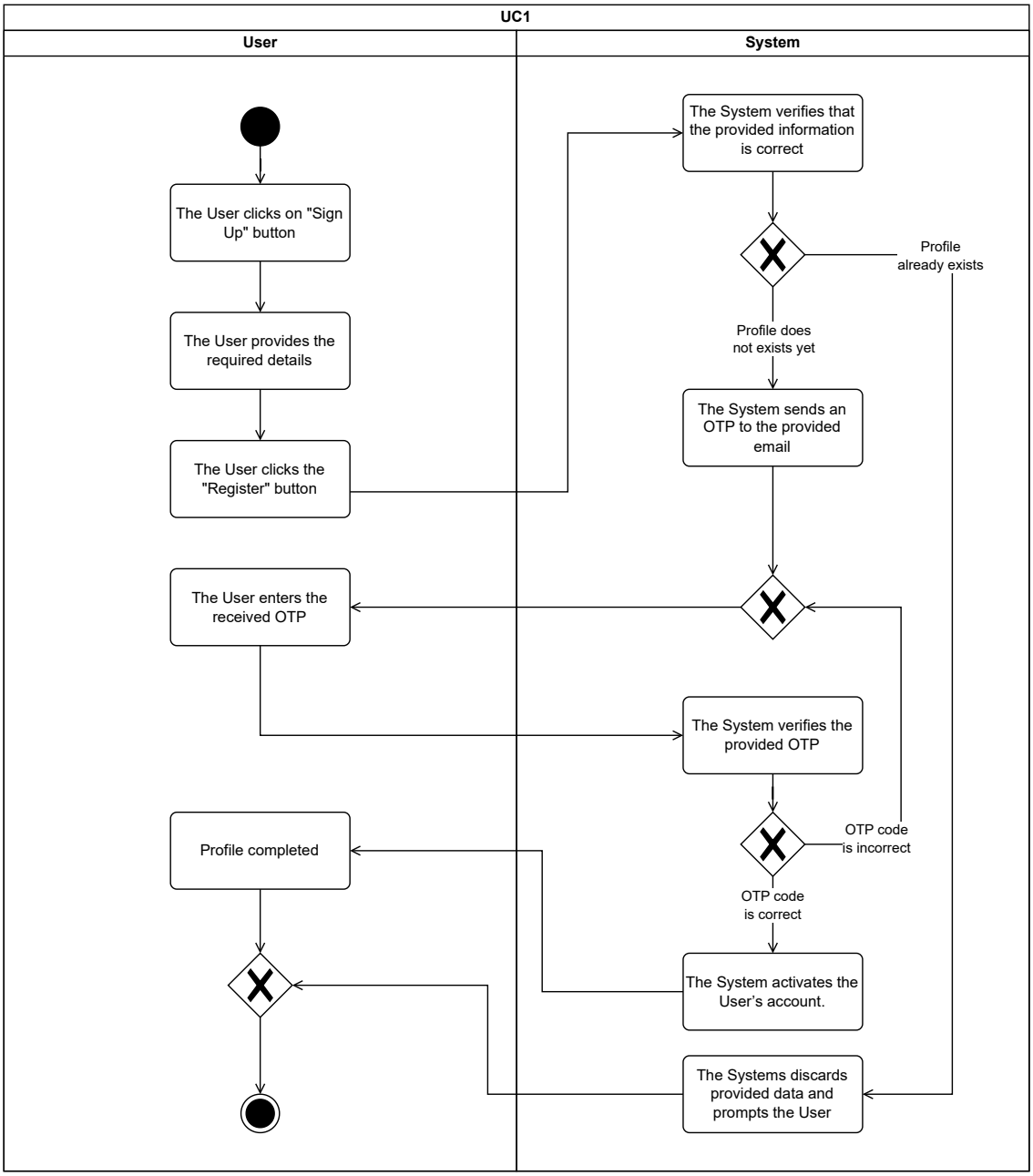


Figure 3.3 – UC1 Activity Diagram

[UC2] Log In by a User

Actor	Registered User
Entry Conditions	The User is not already logged into the software.
Flow of Events	1. On the homepage, the User clicks the *Login* button, which displays the login form. 2. The User enters their email and password into the designated fields. 3. The User clicks the *Login* button. 4. The System validates the provided credentials. 5. The System redirects the User to the *Explore* page.
Exit Conditions	The User is successfully logged in.
Exceptions	• Incorrect credentials: If the credentials provided are incorrect, the System denies access and prompts the User to try again.

Table 3.2 – UC2: Log In by a User

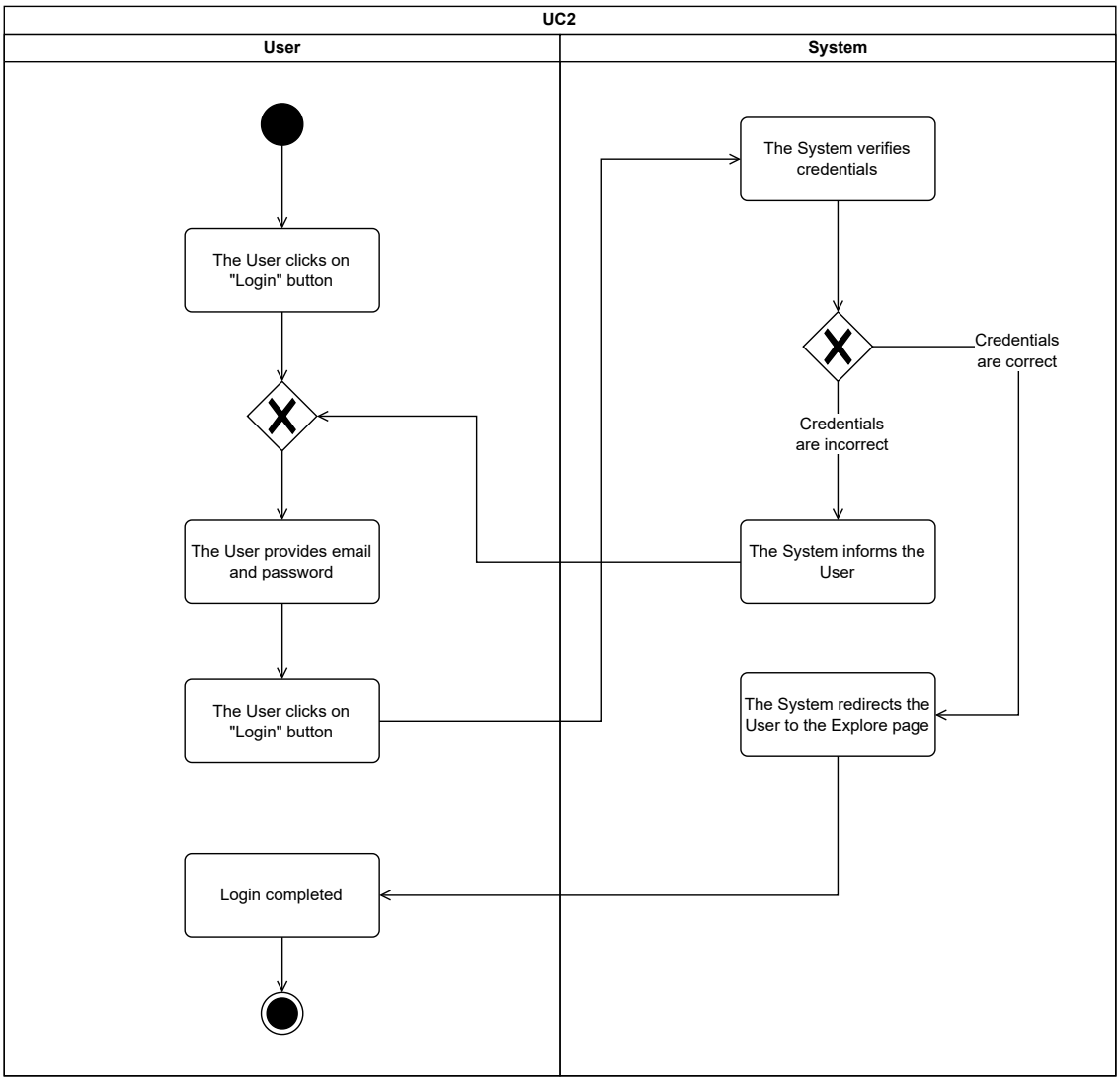


Figure 3.4 – UC2 Activity Diagram

[UC3] Access as Guest

Actor	Unregistered User (Guest)
Entry Conditions	The User has opened the application and is on the landing screen.
Flow of Events	1. On the Login/Sign Up screen, the User selects the *Continue as Guest* option. 2. The System grants access to the public features of the application (Search, Map Visualization). 3. The System restricts access to private features (Recording, My Activity) by hiding them or disabling the buttons. 4. The User is redirected to the *Explore* page.
Exit Conditions	The User can browse and search for Paths but cannot record data.

Table 3.3 – UC3: Access as Guest

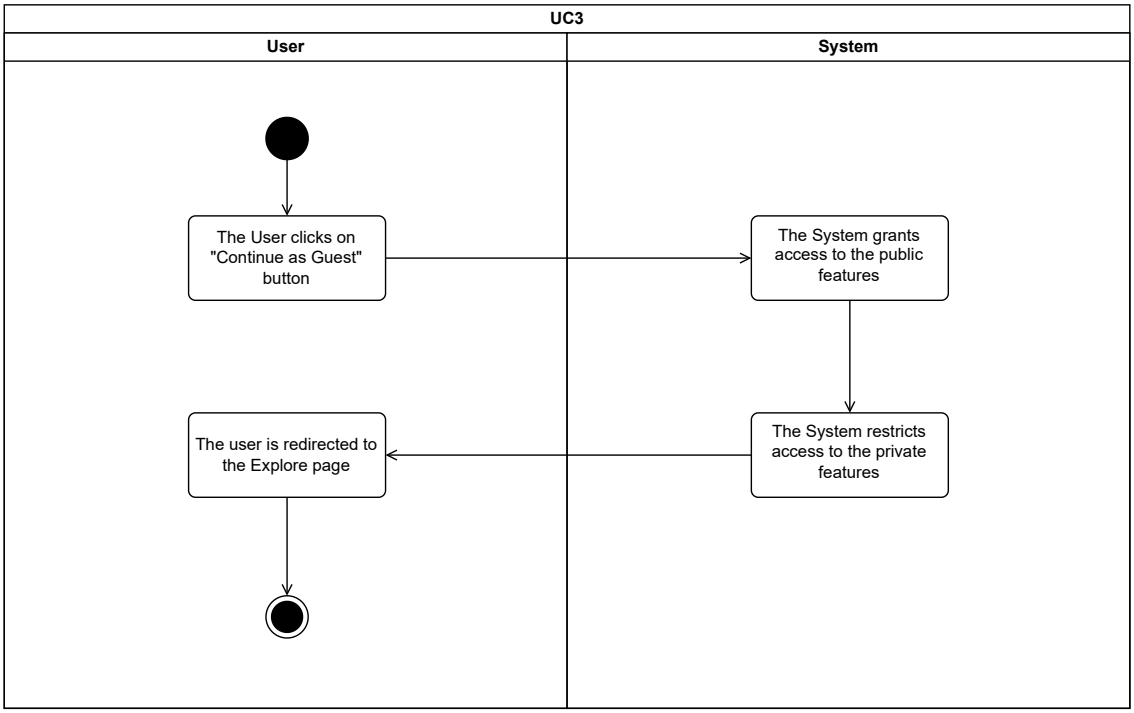


Figure 3.5 – UC3 Activity Diagram

[UC4] Record a Trip

Actor	Registered User
Entry Conditions	The User is logged into the software.
Flow of Events	1. The User selects the *Start Cycling* command to begin a session. 2. The System automatically begins recording GPS coordinates to track the Path. 3. While cycling, the System analyses accelerometer and gyroscope data to automatically detect potential obstacles (e.g., potholes). 4. If the System detects that the User's speed drops below a threshold or movement indicates walking, it automatically pauses data acquisition. 5. The System automatically resumes recording when the User starts pedalling again. 6. The User selects the *End Cycling* command to end the session. 7. The System retrieves weather data from an external service associated with the location and time of the Trip. 8. The System calculates performance statistics (speed, duration, distance). 9. The System presents a Trip summary and prompts the User to validate any detected obstacles.
Exit Conditions	The Trip has been recorded, the statistics are calculated, and the Path is ready for User validation.
Exceptions	• Non-Cycling Activity: If the User never cycles (e.g., only walks), the System pauses recording and the Segment is not added to the cycling Path inventory.

Table 3.4 – UC4: Record a Trip

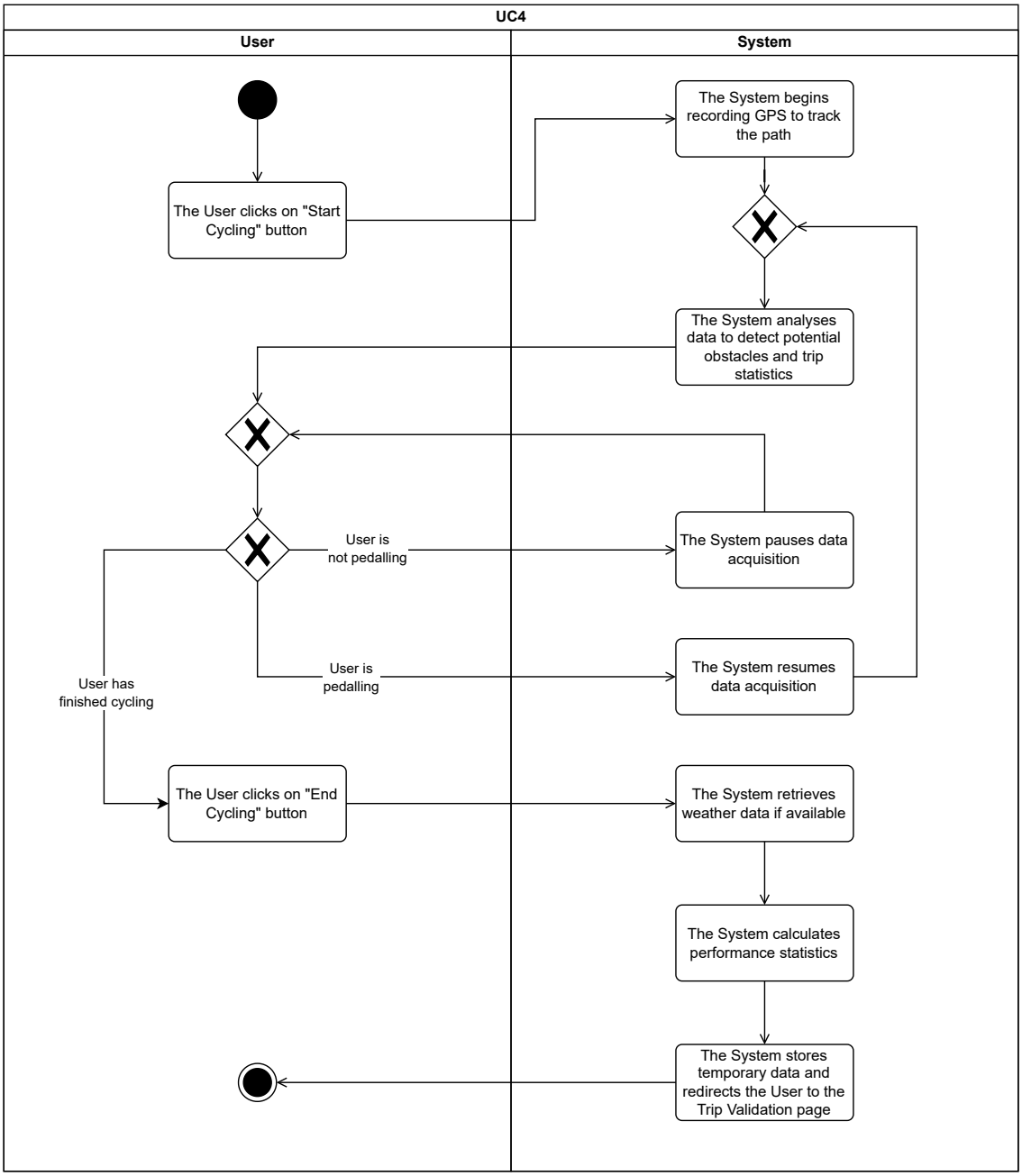


Figure 3.6 – UC4 Activity Diagram

[UC5] Validate a Trip and Path

Actor	Registered User
Entry Conditions	The User is logged into the software and they have just finished a recording session (pressed *End Cycling*).
Flow of Events	1. The System shows the User a Trip summary and any automatically detected obstacles. 2. The User examines the proposed obstacles. 3. The User confirms the presence of real obstacles or rejects false positives (swiping or tapping on the list). 4. The User manually inserts any additional obstacles that the System failed to detect, pinpointing them on the map and assigning a type (e.g., *Pothole*). 5. The User assigns a general status to the Path just completed (e.g., *Optimal, Needs Maintenance*). 6. The System applies this status to all the road Segments traversed during the Trip and, 7. Optionally, the User selects specific Segments on the map to override the general status with a more specific one (e.g., marking only one section as *Needs Maintenance*). 8. The User confirms saving the Trip and the Path and decides whether to make the Path public or private. 9. The System permanently saves the Trip, the Path (linking the specific Segment statuses) and confirmed data in the database, preventing future changes to the GPS track. If the User opted for the publication of the Path, the System launches the aggregation algorithm: it recalculates the status of the affected road Segments by merging the User's new data with the community's existing data.
Exit Conditions	The Trip is saved and the Path is validated and saved in the User's profile; the obstacle information has been cleaned of false positives.
Exceptions	• Trip cancellation: If the User realizes that the recording is invalid, they decide to delete the entire Trip (Path included) instead of validating it. The System removes all temporary data and nothing is saved.

Table 3.5 – UC5: Validate a Trip and Path

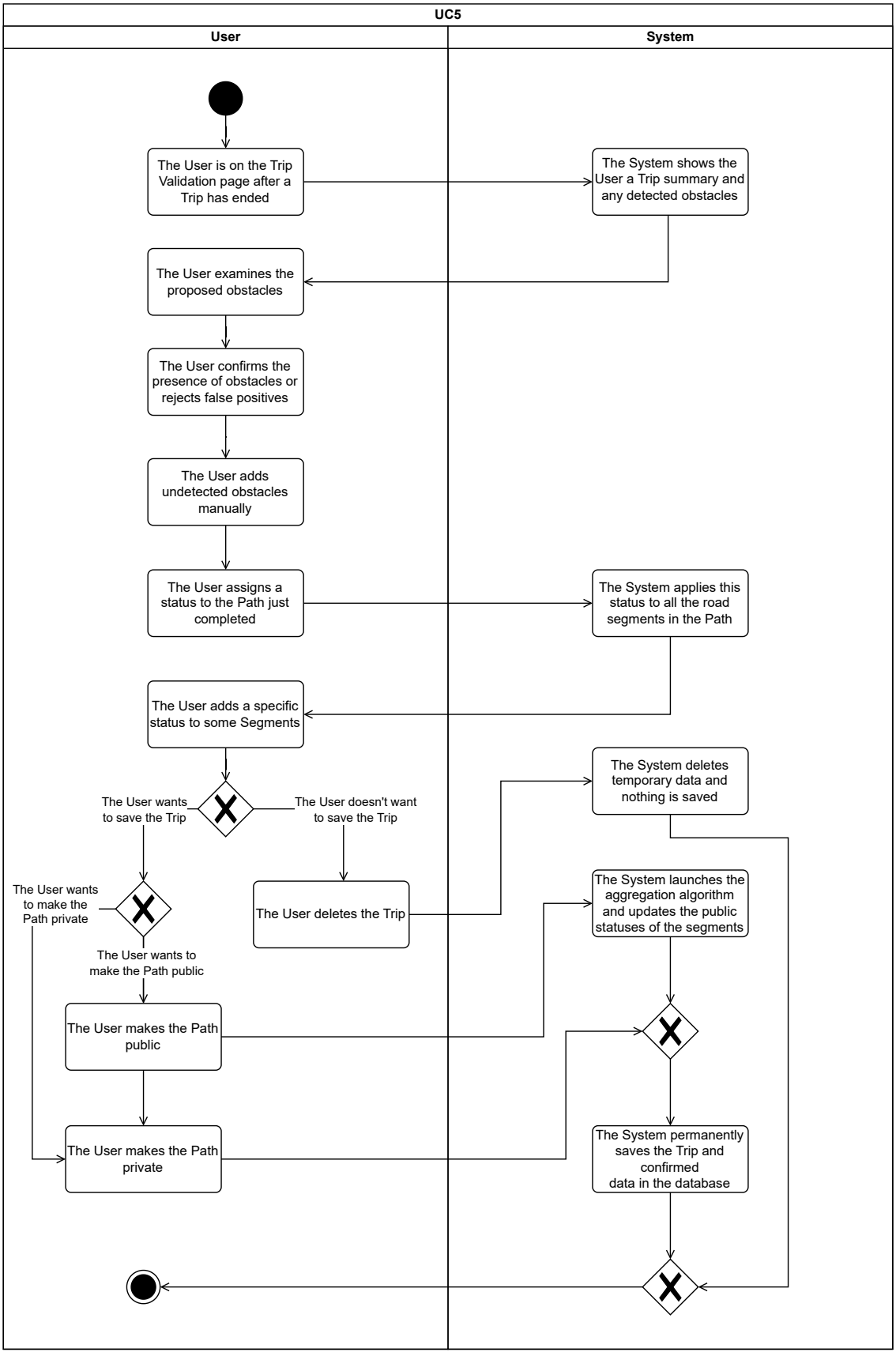


Figure 3.7 – UC5 Activity Diagram

[UC6] Change Path's visibility

Actor	Registered User
Entry Conditions	The User has inserted a Path, either manually or automatically, and is in the Path details page.
Flow of Events	1. The User chooses the option to make the Path *Public*. 2. The System receives the request and updates the Path's visibility attribute. 3. The System launches the aggregation algorithm: it recalculates the status of the affected road Segments by merging the User's new data with the community's existing data. 4. The System resolves any data conflicts based on the freshness of the information and the number of confirmed reports. 5. The System confirms to the User that the Path has been successfully published.
Exit Conditions	The Path is visible to all Users on the global map and aggregated community data (scores, road status) has been updated.
Exceptions	The Path was already public: the System will not perform any operation.

Table 3.6 – UC6: Change of Path's visibility

Note that the described procedure outlines the process for making a Path public. However, it is also possible to revert a previously published Path to a private state in the same way. In such a case, the System would set the visibility status to *private* and all related aggregated data would be restored.

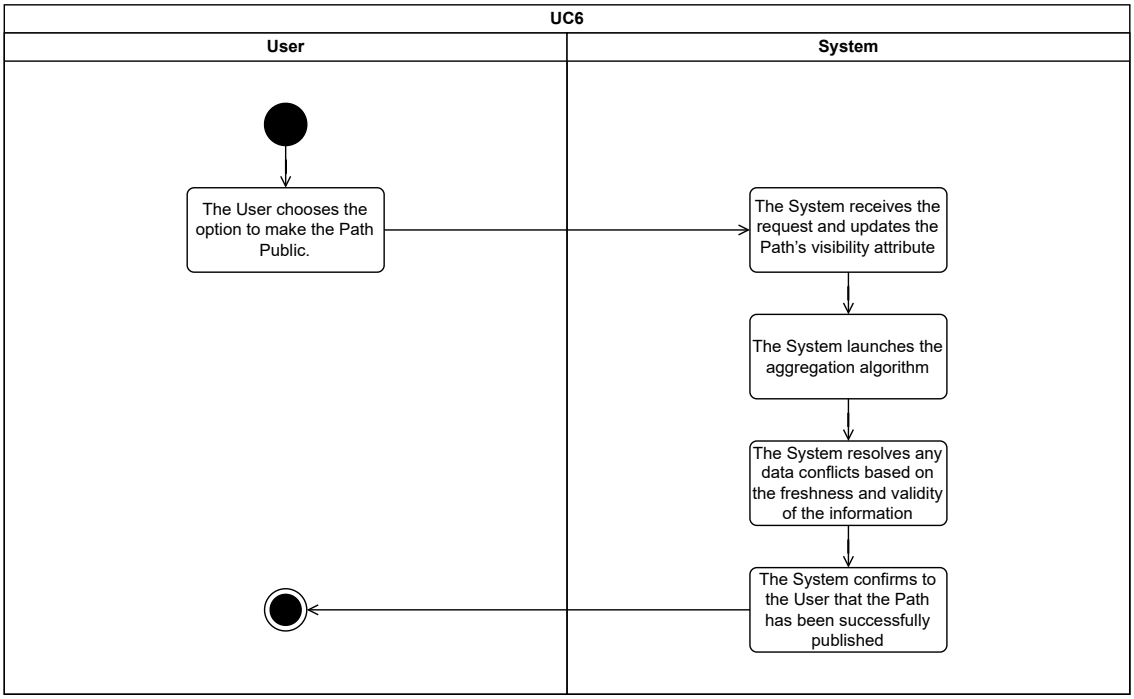


Figure 3.8 – UC6 Activity Diagram

[UC7] Create a Path manually

Actor	Registered User
Entry Conditions	The User is logged into the software and they have selected the *Create a Path* functionality.
Flow of Events	1. The User interacts with the map and manually enters the sequence of road Segments connecting an entry and exit point. 2. The User assigns a general status to the Path (e.g., *Optimal*, *Hard*) to warn future cyclists. 3. The System applies this status to all selected Segments. 4. Optionally, the User selects specific Segments on the map to override the general status with a more specific one (e.g., marking only one section as *Needs Maintenance*). 5. The User sets the Path visibility (public/private). 6. The User confirms Path creation. 7. The System validates that a list of roads and statuses has been provided. 8. The System saves the Path in the database. 9. If set to public, the System immediately updates the global map by recalculating the aggregated status of the involved Segments.
Exit Conditions	The new Path has been saved in the System and, if public, is integrated into the global inventory of cycling Paths.
Exceptions	• Incomplete Data: If the User attempts to save without providing the road list or status, the System prevents saving and requests the missing data. • Mapping Error: If the selected Segments are not contiguous or valid on the map, the System reports a tracking error.

Table 3.7 – UC7: Create a Path manually

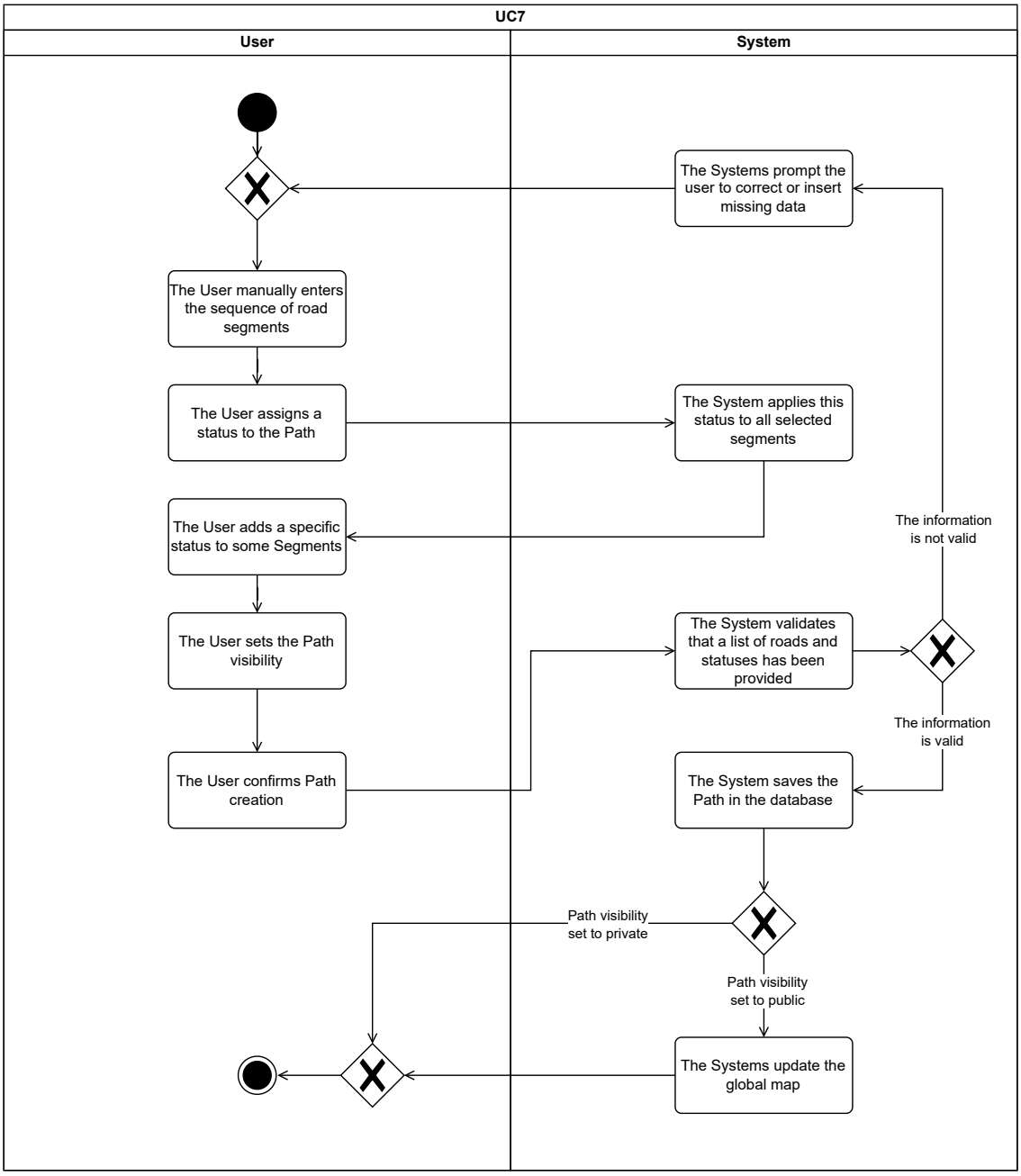


Figure 3.9 – UC7 Activity Diagram

[UC8] Search for a Path

Actor	Guest User, Registered User
Entry Conditions	The User has access to the application and they have selected the search section.
Flow of Events	1. The User enters a source point in the search form. 2. The User enters a destination point in the search form. 3. The System searches the database for possible Paths connecting the two points. 4. If multiple Paths exist, the System calculates a score for each, based on the aggregate status (e.g., presence of potholes) and the efficiency/distance to the destination. 5. The System displays the results on a map, sorting or filtering them based on the calculated score. 6. The User views the alternatives and selects one for details.
Exit Conditions	The User sees a list of valid Paths between the specified points, with their statuses and scores.
Exceptions	• No Path found: If there are no mapped cycling Paths between the origin and destination (or if they are not connected), the System informs the User that a Path cannot be found.

Table 3.8 – UC8: Search for a Path

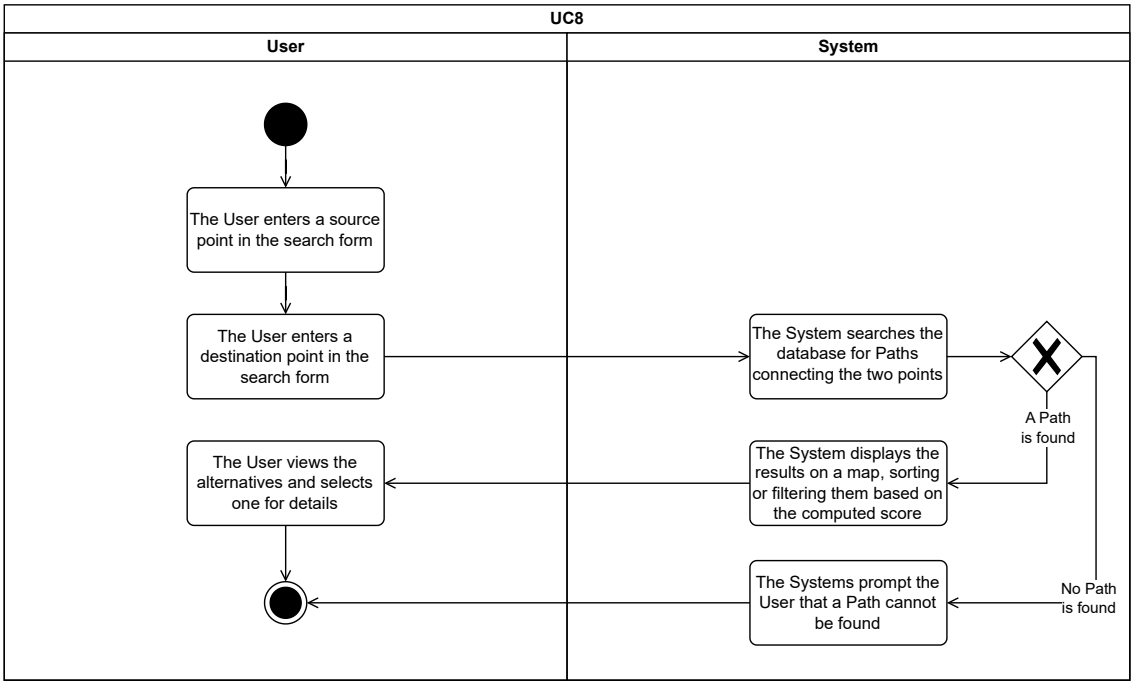


Figure 3.10 – UC8 Activity Diagram

[UC9] Delete a Trip

Actor	Registered User
Entry Conditions	The User is logged in and has at least one recorded Trip in the *My Trips* section.
Flow of Events	1. The User views the details of a specific Trip. 2. The User presses the *Delete* button. 3. The System asks the User to choose the scope of the deletion: • Option A: Delete the Trip (statistics, time) but keep the Path (geometry/map). • Option B: Delete both the Trip and the associated Path. 4. The User selects an option and confirms the action. 5. **Case A (Keep Path):** The System removes the Trip data (speed, duration) but retains the Path in the User's *My Paths* list as a manually saved Path. 6. **Case B (Delete Both):** The System removes both the Trip and the Path. If the Path was public, the System removes the User's contribution from the aggregated community data.
Exit Conditions	The Trip is removed. The associated Path is either preserved as an orphan entry or permanently deleted based on User choice.
Exceptions	• User Cancellation: If the User cancels the confirmation prompt, no data is modified.

Table 3.9 – UC9: Delete a Trip

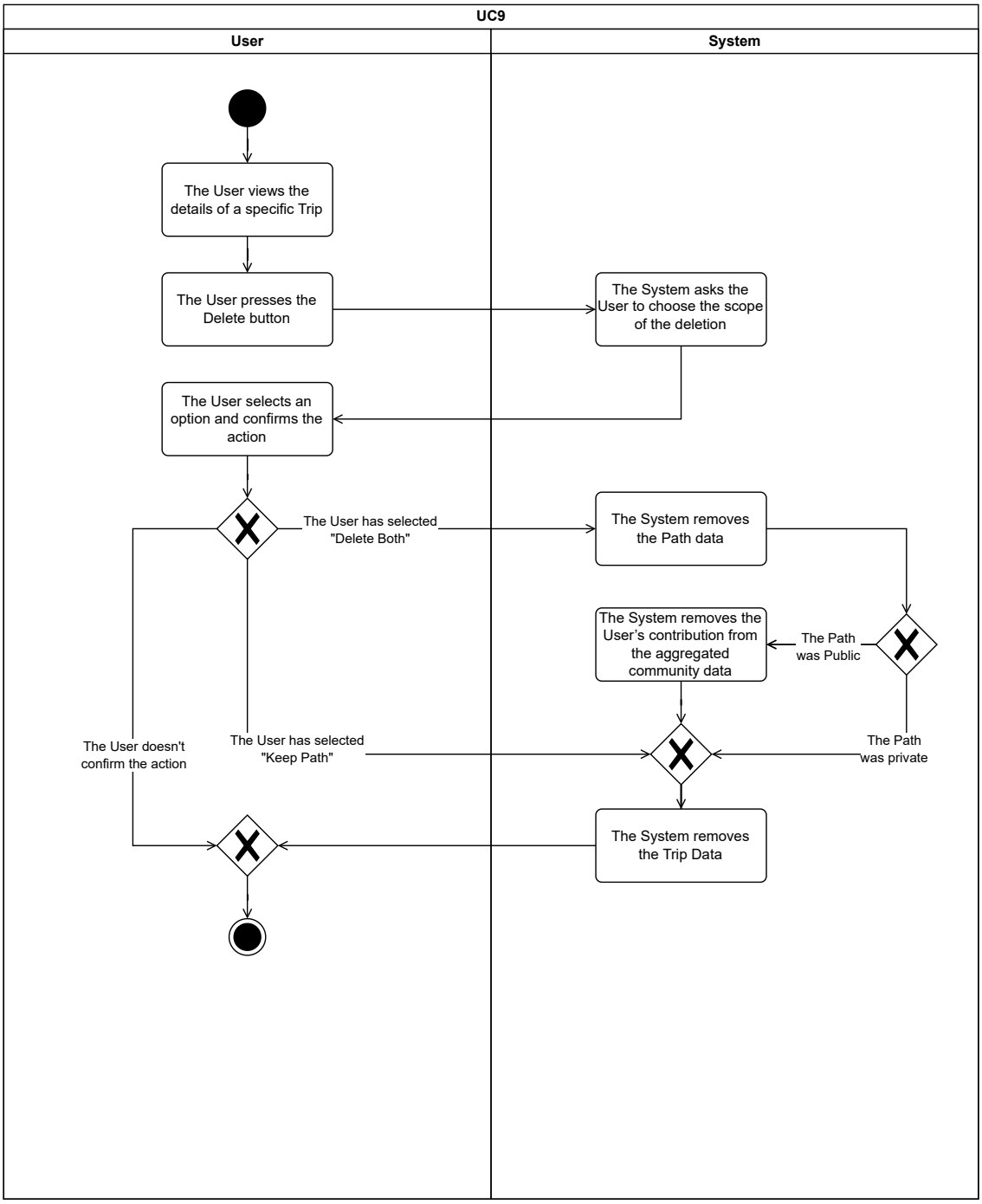


Figure 3.11 – UC9 Activity Diagram

[UC10] Delete a Path

Actor	Registered User
Entry Conditions	The User is logged in and has at least one Path in the *My Paths* section.
Flow of Events	1. The User selects the specific Path they wish to remove. 2. The User presses the *Delete* button. 3. The System checks for dependencies: it verifies if there is any existing recorded *Trip* currently linked to this *Path*. 4. If no dependencies are found (the Path is *orphan* or manually created), the System prompts for confirmation. 5. The User confirms the operation. 6. The System permanently removes the Path from the User's profile. 7. If the Path was public, the System triggers the aggregate data update process to remove this contribution.
Exit Conditions	The Path is deleted from the System and removed from the global map aggregation if it was public.
Exceptions	• Dependency Error: If the System finds a *Trip* associated with the Path, it blocks the deletion and displays an error message. • User Cancellation: If the User denies confirmation, the Path remains intact.

Table 3.10 – UC10: Delete a Path

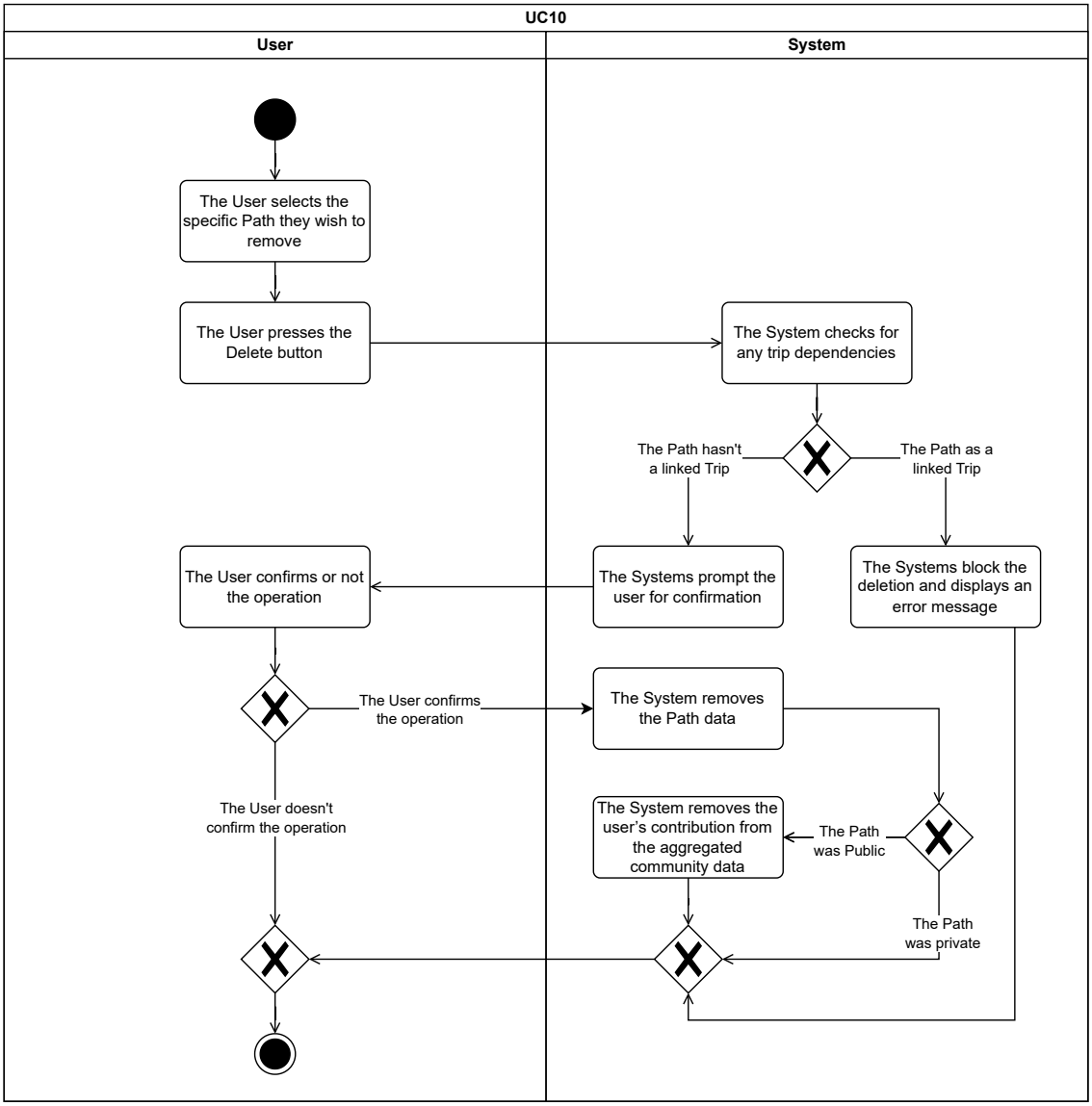


Figure 3.12 – UC10 Activity Diagram

3.2.3 Requirement Mapping

UC ID	Goals	Requirements	Assumptions
UC1	G2, G4, G8	R1	DA3, DA10
UC2	G1, G2, G3, G4, G5, G8	R2	DA10
UC3	G6, G7	R3	DA10
UC4	G1, G2, G3, G5	R4, R5, R6, R7, R8, R9, R23, R24	DA1, DA2, DA6, DA7, DA9, DA10
UC5	G2, G5, G6	R10, R11, R12	DA1, DA5, DA10
UC6	G6, G8	R16, R17, R19, R20, R21	DA4, DA5, DA10
UC7	G4, G8	R13, R19	DA4, DA5, DA8, DA10
UC8	G6, G7	R25, R26	DA2, DA8, DA10
UC9	G2, G6	R14, R18, R22	DA10
UC10	G4, G6	R15, R18, R22	DA10

Table 3.11 – Requirements Mapping

3.3 Performance Requirements

Given the mixed nature of the System (real-time sensing on the client, heavy data aggregation on the server), the performance requirements are defined as follows:

- **Response Time for Interactive Tasks**: For standard User interactions (e.g., Log In, accessing *My Trips*, viewing a static map), the System shall respond to the User request within **1 second** under normal load conditions (excluding network latency).
- **Path Search Latency**: Due to the computational complexity of the routing algorithm (which factors in scores, status, and distance), the System shall provide a list of suggested Paths between two locations within **5 seconds** for 95% of requests.
- **Sensor Processing Latency (Client-side)**: To ensure the spatial accuracy of detected obstacles, the mobile application shall process accelerometer and gyroscope variations and associate them with the current GPS coordinate with a latency not exceeding **200 milliseconds**.
- **Data Aggregation Freshness**: The *Merging Algorithm* responsible for updating Path statuses based on new User reports shall run continuously or in short bursts. The System shall ensure that a validated report from a User is reflected in the global public statistics (and thus visible to other Users) within **10 minutes** of the data upload.
- **System Capacity and Scalability**: The backend infrastructure shall support at least **10,000 concurrent Users** (recording Trips or searching for Paths) without degradation of the response times specified above. The architecture must be horizontally scalable to handle seasonal peaks (e.g., spring/summer weekends).
- **Synchronization Latency**: Upon restoration of internet connectivity following a period of offline activity, the application shall automatically synchronize locally cached data with the central server. The upload of a typical 1-hour Trip log (including GPS sequence, sensor metadata, and timestamps) must complete in less than **30 seconds** over a standard 4G/LTE connection.

3.4 Design Requirements

3.4.1 Standards compliance

The System is designed to adhere to the following standards and regulations to ensure legal compliance, User privacy, and store eligibility:

1. **General Data Protection Regulation (GDPR)**: As the Platform processes sensitive geolocation and sensor data, strict adherence to GDPR is enforced. Key measures include:
    - **Data Anonymization**: Removing personal identifiers from Trip data before aggregation.
    - **Data Minimization**: Collecting only the data strictly necessary for the service provision.
    - **User Rights**: Mechanisms for Users to access, rectify, and delete their data (Right to be Forgotten).
    - **Consent**: Explicit and granular consent collection processes for background tracking.
    - **Transparency**: Clear data usage policies accessible within the app.
2. **Apple App Store Review Guidelines**: The iOS application must comply with Apple's strict standards to ensure approval. Focus areas include:
    - **Safety (User Generated Content)**: Implementation of filtering or reporting mechanisms for offensive or malicious Path descriptions created by Users.
    - **Privacy (Location Services)**: Compliance with guidelines for requesting background location access only when necessary and with a clear visual indicator.
    - **Performance**: Ensuring the app does not excessively drain the battery or generate excessive heat during tracking.
3. **Google Play Developer Policy**: The Android application must adhere to Google's policies, specifically:
    - **Permissions Policy**: Justifying the use of sensitive permissions as core functionality.
    - **User Data**: Secure handling of personal information and prominent disclosure of data access.
    - **Monetization and Ads**: Compliance with policies regarding any future promotional content, ensuring it does not interfere with app usability.
4. **Web Content Accessibility Guidelines (WCAG) 2.1**: To ensure inclusivity for Users with disabilities, the mobile interface design follows the W3C principles where applicable to mobile contexts:
    - **Perceivable**: Providing text alternatives for non-text content (e.g., map icons) and sufficient colour contrast ratios (Level AA).
    - **Operable**: Ensuring all functionalities are accessible via assistive technologies (e.g., VoiceOver, TalkBack).
    - **Understandable**: Using clear navigation labels and readable fonts that support dynamic resizing.

3.4.2 Hardware limitations

The design and implementation of the BBP mobile application must account for the physical and technical constraints inherent to mobile devices:

- **Battery Consumption**: Continuous access to the GPS module and high-frequency sampling of the accelerometer and gyroscope are highly energy-intensive operations. To prevent rapid battery drainage during long cycling sessions, the application must implement optimization strategies. These include:
    - Adaptive sampling rates (e.g., reducing sensor polling frequency when the User is stationary or paused).
    - Efficient background processing to minimize CPU wake-locks.

- **Sensor Accuracy and Heterogeneity**: The quality, sensitivity, and calibration of hardware sensors (accelerometer, gyroscope) vary significantly across different device models and manufacturers. The obstacle detection algorithm must be robust enough to handle signal noise and varying hardware specifications without generating excessive false positives. Furthermore, GPS accuracy may degrade in specific environments (e.g., dense forests, urban canyons), requiring software-side compensation (e.g., map matching) to ensure Path coherence.
- **Local Storage Capacity**: To support the caching of the Path while cycling, the application relies on the device's internal storage. The System must implement efficient data compression and cache management policies to avoid exceeding reasonable storage quotas on User devices with limited capacity.
- **Thermal Constraints**: Intensive processing of sensor data combined with screen usage under direct sunlight (typical for cycling) can lead to device overheating. The application logic must be optimized to minimize computational load, preventing the operating System from throttling performance or shutting down the app for thermal protection.

3.4.3 Any other constraint

- **Network Connectivity and Volatility**: As established in the Domain Assumptions, the application is frequently used in environments with unstable or absent cellular coverage (e.g., rural areas, mountain trails). Therefore, the System architecture involves a constraint on data synchronization: non-critical operations (e.g., uploading Trip data, downloading map updates) must be deferrable and capable of resuming automatically when connectivity is restored (*Store and Forward* mechanism).
- **Third-Party API Rate Limits**: The System relies on external services for map tiles and weather data. To maintain operational sustainability and avoid service denial due to exceeding quota limits (Rate Limiting), the System must implement strict caching policies (e.g., caching weather data for a specific area for 1 hour instead of refreshing it at every request).
- **Safety and Liability Disclaimer**: Since the application suggests physical Paths and relies on User generated content which might be outdated or subjective, the System interface is constrained to display a mandatory safety disclaimer upon startup. The System must explicitly inform Users that Path suggestions are for informational purposes only and do not replace adherence to local traffic laws or personal judgment of road conditions.

3.5 Software System Attributes

3.5.1 Reliability

To guarantee dependable performance, the System must capably manage errors and rapidly recover from interruptions, thereby reducing overall downtime, the frequency of issues, and the impact on specific features. This requirement shall be fulfilled by utilizing **industry-standard fault-tolerance protocols** (specifically through the **redundancy** of data storage and processing nodes) to **eliminate single points of failure**.

3.5.2 Availability

The backend System must maintain a **minimum uptime of 99%** (2-nines), limiting unscheduled outages to a maximum of 3.65 days annually. The server-side service must remain accessible at all times, with the exception of planned maintenance windows scheduled during off-peak hours and communicated in advance. However, regarding the client-side mobile application, the System shall allow Users to record Trips and acquire sensor data even in the **absence of internet connectivity**, **locally caching** the information and **synchronizing** it with the central server once the connection is restored. This ensures the primary function (tracking) is available even in remote areas.

3.5.3 Security

The System is required to implement **secure authentication** using email and password credentials, alongside **authorization mechanisms** to verify identities and restrict feature access according to User roles. These controls shall be enforced through rigorous password policies and encrypted communication protocols such as **HTTPS**. Furthermore, the Platform must guarantee **data integrity** by prohibiting unauthorized modifications to preserve the accuracy of information. To ensure **confidentiality**, sensitive data must be **encrypted both in transit and at rest**, rendering it unreadable in the event of interception or exposure. Additionally, the System must remain resilient against standard attack vectors, including **SQL injection, XSS, and CSRF**, while incorporating regular security patches to mitigate new vulnerabilities.

3.5.4 Maintainability

A maintenance schedule must be established to accommodate hardware/software upgrades, new feature deployment, bug resolution, and security patching. These operations are expected to average 8 hours in duration, targeting **off-peak traffic windows**, and must strictly adhere to a **24-hour maximum limit**. Major maintenance events are to be considered exceptions, occurring no more than two to three times annually, and require a public announcement at least 48 hours prior. Furthermore, software maintainability shall be ingrained in the development lifecycle from the outset through the enforcement of industry best practices. The resulting codebase must be **clean, modular, reusable, and loosely coupled** to streamline the future integration of new functionalities.

3.5.5 Portability

The System is designed as a **Cross-Platform Mobile Application**, ensuring compatibility with the two major mobile operating Systems (**Android and iOS**) through a **unified codebase** (e.g., using frameworks like Flutter or React Native). This approach guarantees a consistent User experience across different devices while minimizing development and maintenance efforts compared to maintaining two separate native codebases. Furthermore, the choice of a mobile app architecture over a web-based one is dictated by the requirement to access **native hardware features**, specifically the accelerometer, gyroscope, and GPS background services, with high performance and reliability, which are essential for the automated obstacle detection features.

Chapter 4

Formal Analysis Using Alloy

4.1 The Model Structure

The formal Model has been developed using **Alloy 6**, leveraging its support for Linear Temporal Logic (LTL) to verify the dynamic behaviour of the System. The code is organized into modular files to enhance readability and maintainability: signatures, predicates, facts, assertions, and a main execution module. The primary goal of this Model is to verify the correctness of the algorithm responsible for updating Segment status upon the publication of a Path, ensuring data consistency and isolation between Users.

4.1.1 Signatures

This module defines the structural atoms and the state of the System. A *var* keyword is used to model mutable elements that evolve over time steps. The representation abstracts away certain details to focus on the core verification goals:

- **Users:** Only registered Users are modelled, as Guest Users operate in a read-only mode that does not alter the System state.
- **Obstacles:** These are omitted as they are static attributes in this context and do not influence the dynamic propagation of visibility and status logic.
- **PathEntries:** These are abstracted away. In the domain model, a Path is composed of entries; in this formal Model, the Path's status is applied directly to the set of involved Segments. This abstraction simplifies the state space without compromising the logic of the propagation algorithm.

```
module signatures

// Simplified enumerations that emulate the different statuses
enum Visibility { Private, Public }
enum Status { Optimal, NeedsMaintenance, Dangerous, Unknown }
enum Weather { Sunny, Cloudy, Rainy }

// Segments have a status that is influenced by the publication of Paths
var sig Segment {
   var currentStatus: Status
}

// Guests are not represented in the Model as their role is to view data
// Each User has a set of Trips and Paths that can grow over time
var sig RegisteredUser {
   var trips: set Trip,
   var paths: set Path
}

// Paths have at leat one Segment, a status and a visibility. Their visibility affects Segment
status
var sig Path {
   var segments: some Segment,
   var status: one Status,
   var visibility: Visibility
}

// A Trip is associated with exactly one Path
var sig Trip {
   var path: one Path,
   var weather: lone Weather
}
```

4.1.2 Predicates

This module outlines the possible state transitions (operations) within the System. A representative subset of actions has been selected to model the critical lifecycle of Paths and Trips. Operations such as account deletion or unpublishing a Path are excluded from this abstraction. This design choice allows the Model to focus specifically on the *safety* and *liveness* properties of the publication workflow without the complexity of edge cases that do not affect the core update algorithm. Each predicate defines strict pre-conditions and post-conditions to guide the solver.

```
module predicates

open signatures

pred initialize {
   // For initial setup
   no RegisteredUser
   no Trip
   no Path
   no Segment
}

pred doNothing {
   // Does nothing
}

pred signUp {
   // Transition: Introduces a new RegisteredUser atom.
   // Post-condition: New user has empty relation sets.
   some newUser: RegisteredUser' - RegisteredUser | {
      RegisteredUser' = RegisteredUser + newUser
      no newUser.trips'
      no newUser.paths'
   }
}

pred recordTrip [u: RegisteredUser, st: Status, w: Weather] {
   // Pre-condition: User is registered.
   u in RegisteredUser

   some recordTrip: Trip' - Trip, newPath: Path' - Path | {
      // Universe expansion
      Trip' = Trip + recordTrip
      Path' = Path + newPath
      Segment in Segment'

      let newSegs = Segment' - Segment {
         // New segments are strictly included in the new path.
         newSegs in newPath.segments'

         // Dynamic expansion: At least 1 new segment must be created.
         #newSegs >= 1

         // Complexity constraint: Path must consist of at least 2 segments.
         #newPath.segments' >= 2

         // Property assignment
         recordTrip.path' = newPath
         recordTrip.weather' = w
         newPath.userRating' = st
         newPath.visibility' = Private

         // Relation updates
         trips' = trips + (u -> recordTrip)
         paths' = paths + (u -> newPath)

         // New segments default to Unknown status.
```

```
            currentStatus' = currentStatus + (newSegs -> Unknown)
        }
    }
}

pred createManualPath [u: RegisteredUser, st: Status] {
    // Pre-condition: User is registered.
    u in RegisteredUser

    some newPath: Path' - Path | {
        // Universe expansion
        Path' = Path + newPath
        Trip' = Trip
        Segment in Segment'

        let newSegs = Segment' - Segment {
            // New segments are strictly included in the new path.
            newSegs in newPath.segments'

            // Dynamic expansion: At least 1 new segment must be created.
            #newSegs >= 1

            // Complexity constraint: Path must consist of at least 2 segments.
            #newPath.segments' >= 2

            // Property assignment
            newPath.segments' = newSegs
            newPath.userRating' = st
            newPath.visibility' = Private

            // Relation updates
            paths' = paths + (u -> newPath)
            currentStatus' = currentStatus + (newSegs -> Unknown)
        }
    }
}

pred publishPath [u: RegisteredUser, p: Path] {
    // Pre-conditions
    u in RegisteredUser
    p in u.paths
    p.visibility = Private

    // Target path visibility becomes Public.
    p.visibility' = Public

    // Visibility of all other paths remains invariant.
    all otherP: Path - p | otherP.visibility' = otherP.visibility

    // Segments update logic.
    currentStatus' = currentStatus ++ (p.segments -> p.userRating)
}
```

4.1.3 Facts

This file contains the Model invariants and consistency conditions. These rules enforce:

- **Structural Integrity:** Constraints that must hold true in every state (e.g., a Path must contain at least one Segment).
- **System Trace:** The definition of allowed temporal evolutions using LTL.
- **Consistency Conditions:** Rules ensuring that variables not explicitly modified by an operation remain unchanged (*inertia*), preventing unwanted side effects.

```
module facts

open signatures
open predicates

fact StructuralRules {
   always {
      // Integrity: No orphaned segments.
      all s: Segment | s in Path.segments

      // Uniqueness: Trip-Path injection.
      all disj t1, t2: Trip | t1.path != t2.path
   }
}

fact OwnershipRules {
   always {
      // Bi-directional consistency between Users, Paths, and Trips.
      all p: Path | one u: RegisteredUser | p in u.paths
      all t: Trip | one u: RegisteredUser | t in u.trips

      // Transitive consistency.
      all u: RegisteredUser, t: u.trips | t.path in u.paths
   }
}

fact NewSegmentsMustBeUnknown {
   always {
      // Initialization rule for disjoint new atoms.
      all s: Segment' - Segment | s.currentStatus' = Unknown
   }
}

fact SystemTrace {
   // Linear Temporal Logic definition of valid traces.
   always {
      initialize
      or doNothing
      or signUp
      or (some u: RegisteredUser, s: Status, w: Weather | recordTrip[u, s, w])
      or (some u: RegisteredUser, s: Status | createManualPath[u, s])
      or (some u: RegisteredUser, p: Path | publishPath[u, p])
   }
}

fact ConsistentTrips {
   always {
      // Internal fields are immutable.
      all t: Trip | t.path' = t.path
      all t: Trip | t.weather' = t.weather

      // Set stability. Trips can be added only through recordTrip[]
      Trip' != Trip implies (some u: RegisteredUser, s: Status, w: Weather | recordTrip[u, s,
      w])
   }
}

fact ConsistentPaths {
   always {
      // Structure and Rating are immutable.
      all p: Path | p.segments' = p.segments
      all p: Path | p.userRating' = p.userRating and p.userRating != Unknown

      // Visibility changes only via publishPath.
      all p: Path |
         p.visibility' != p.visibility
         implies (some u: RegisteredUser | publishPath[u, p])
```

```
      // Set stability. Paths can be added only through recordTrip[] or createManualPath[]
      Path' != Path implies (
         (some u: RegisteredUser, st: Status, w: Weather | recordTrip[u, st, w]) or
         (some u: RegisteredUser, st: Status | createManualPath[u, st])
      )
   }
}

fact ConsistentSegments {
   always {
      // State transition: Segment status synchronizes with Path rating upon publication.
      all s: Segment |
         (s.currentStatus' != s.currentStatus)
         implies (some u: RegisteredUser, p: Path | publishPath[u, p] and s in p.segments and
         s.currentStatus' = p.userRating)

      // Set stability. Segments can be added only through recordTrip[] or createManualPath[]
      Segment' != Segment implies (
         (some u: RegisteredUser, st: Status, w: Weather | recordTrip[u, st, w]) or
         (some u: RegisteredUser, st: Status | createManualPath[u, st])
      )
   }
}

fact ConsistentRegisteredUsers {
   always {
      // User relations update only via specific actions.
      all u: RegisteredUser | u.trips' != u.trips
         implies (some s: Status, w: Weather | recordTrip[u, s, w])

      all u: RegisteredUser | u.paths' != u.paths
         implies (
            (some s: Status, w: Weather | recordTrip[u, s, w])  or
            (some s: Status | createManualPath[u, s])
         )
      // Set stability. Users can be added only through signUp[]
      RegisteredUser' != RegisteredUser implies signUp[]
   }
}
```

4.1.4 Assertions

This module encodes the properties that must be verified against the Model. Rather than representing the full complexity of the real-world implementation, these assertions are tailored to the abstracted Model to formally prove:

- **Safety:** Bad things never happen (e.g., a User cannot publish another User's Path).
- **Liveness:** Good things eventually happen (e.g., Segment status is updated correctly).
- **Possibility:** Valid states are reachable (e.g., Segment sharing between Paths is possible).

```
module assertions

open signatures
open predicates

assert VisibilityIsolation {
   // Verifies that publishing a path does not unintentionally alter the visibility of disjoint
   paths.
   always {
      all u: RegisteredUser, p: Path |
      publishPath[u, p] implies (all otherP: Path - p | otherP.visibility' = otherP.visibility)
   }
```

```
}

assert SegmentsStatusSync {
   // Verifies that segment status changes are strictly causal to path publication.
   always {
      all s: Segment |
      s.currentStatus' != s.currentStatus implies
      (some u: RegisteredUser, p: Path | publishPath[u, p] and s in p.segments)
   }
}

assert OncePublicAlwaysPublic {
   // Verifies that public visibility is an absorbing state.
   always {
      all p: Path | p.visibility = Public implies always (p.visibility = Public)
   }
}

assert SharingIsImpossible {
   // Asserting that segments can NEVER be shared.
   // A counterexample found by the solver proves that segment sharing IS possible.
   always {
      all disj p1, p2: Path | no (p1.segments & p2.segments)
   }
}
```

4.1.5 Main Execution

This file serves as the entry point for the Alloy Analyser. It contains the `run` commands to generate specific trace scenarios and the `check` commands to validate the assertions against the logic defined in the previous modules.

```
module main

open signatures
open predicates
open facts
open assertions

pred largeStaticWorld {
   // Snapshot constraint for population analysis.
   #RegisteredUser = 3
   #Trip = 4
   #Path = 5
   #Segment = 6
   #visibility.Public >= 2
   #currentStatus.Dangerous >= 1
}

// Static World
run largeStaticWorld for 10

pred dynamicWorld {
   // Step 1: Initialization and first sign up.
   (initialize and signUp)

   // Step 2: User 1 records a trip.
   and after (
      (some u: RegisteredUser | recordTrip[u, Optimal, Sunny])

      // Step 3: User 2 signs up.
      and after (
         signUp

         // Step 4: User 2 creates a manual path.
         and after (
```

```
            (some u: RegisteredUser | #u.paths = 0 and createManualPath[u, Dangerous])

            // Step 5: User 1 publishes their path.
            and after (
               (some u: RegisteredUser, p: Path | p.userRating = Optimal and publishPath[u, p])

               // Step 6: User 3 signs up.
               and after (
                  signUp

                  // Step 7: User 3 records a trip.
                  and after (
                     (some u: RegisteredUser | #u.paths = 0 and recordTrip[u, NeedsMaintenance,
                     Cloudy])

                     // Step 8: User 2 publishes their path.
                     and after (
                        (some u: RegisteredUser, p: Path | p.userRating = Dangerous and
                        publishPath[u, p])

                        // Step 9: User 3 publishes their path.
                        and after (
                           (some u: RegisteredUser, p: Path | p.userRating = NeedsMaintenance
                           and publishPath[u, p])
                     )
                     )
                  )
               )
            )
         )
      )
   )
}

// Dynamyc World
run dynamicWorld for 5 but 15 Segment

// Verification Commands
check VisibilityIsolation for 5
check SegmentsStatusSync for 5
check OncePublicAlwaysPublic for 5
check SharingIsImpossible for 5
```

4.2 Validation through the Model

This section illustrates the validation results obtained from the Alloy Analyzer. Instances generated by the `run` commands demonstrate the consistency of the Model, while the assertion checks verify the safety and liveness properties of the System.

4.2.1 Large Static World

The execution of the `run largeStaticWorld` command successfully produced a consistent instance, confirming that the structural axioms allow for a populated System state without contradictions. Figure 4.1 depicts a meaningful snapshot of the System generated by the Analyser, featuring multiple Users, Paths, and Segments satisfying the multiplicity constraints.

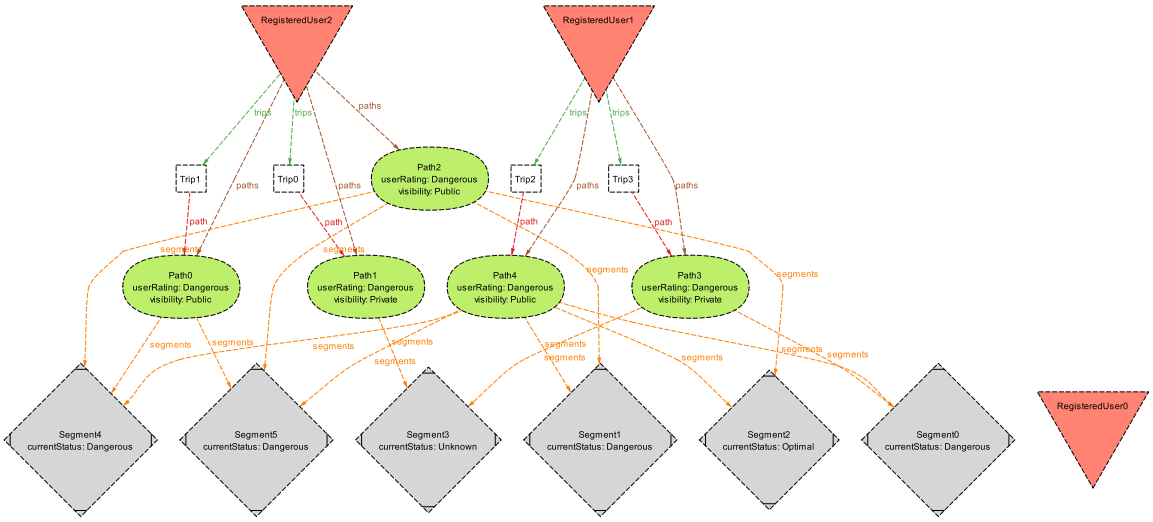


Figure 4.1 – Large Static World Instance

4.2.2 Dynamic World

The `run dynamicWorld` command generates a trace satisfying the Linear Temporal Logic (LTL) specifications defined in the *SystemTrace* fact. The following figures illustrate the step-by-step evolution of the System state.

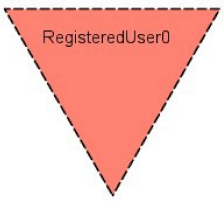


Figure 4.2 – Dynamic World, Step 1: System Initialization and Sign Up

The trace begins in Figure 4.2 with the initialization of the universe and the `signUp` of the first User.

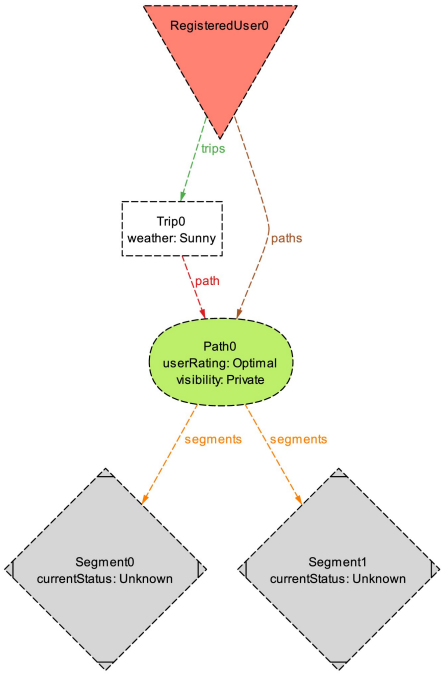


Figure 4.3 – Dynamic World, Step 2: Trip Recording

In Figure 4.3, User 0 records a Trip. This action triggers the creation of a new Path and the instantiation of the corresponding Segments.

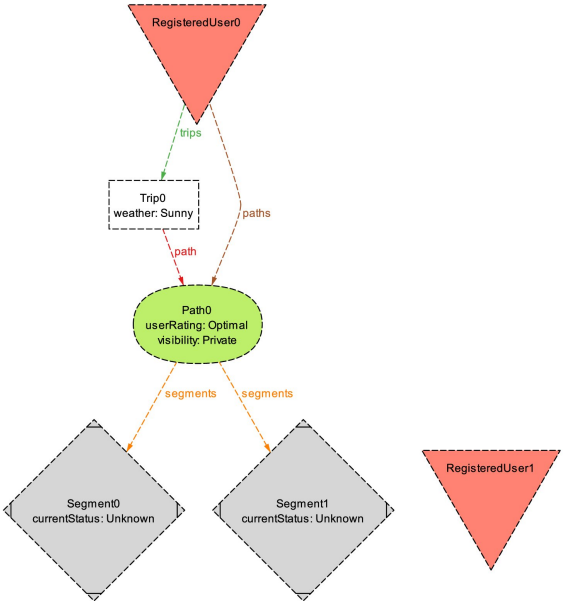


Figure 4.4 – Dynamic World, Step 3: User 1 Sign Up

Figure 4.4 shows the expansion of the User base with the registration of User 1.

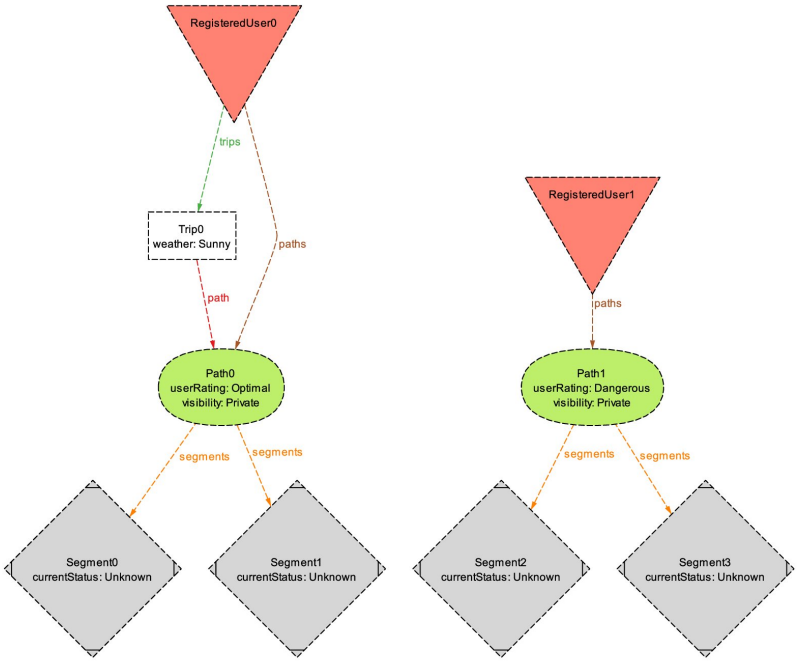


Figure 4.5 – Dynamic World, Step 4: Manual Path Creation

In Figure 4.5, User 1 creates a manual Path. At this stage, the Path is private and its Segments are initialized with an *Unknown* status.

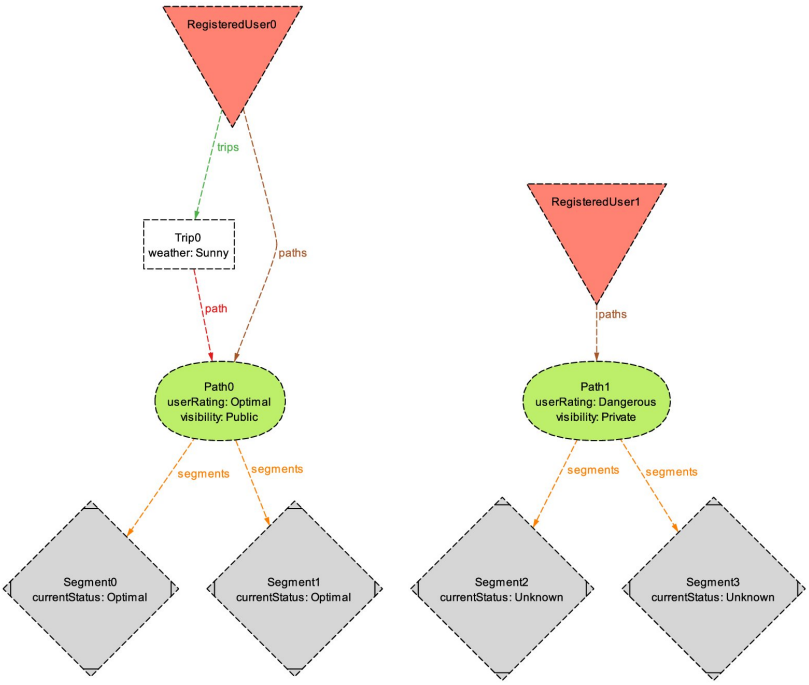


Figure 4.6 – Dynamic World, Step 5: Path Publication

Figure 4.6 illustrates the publication of User 0's Path. It is crucial to observe the side effect on the involved Segments: their status is updated from *Unknown* to the rating provided by the User, propagating the information to the System.

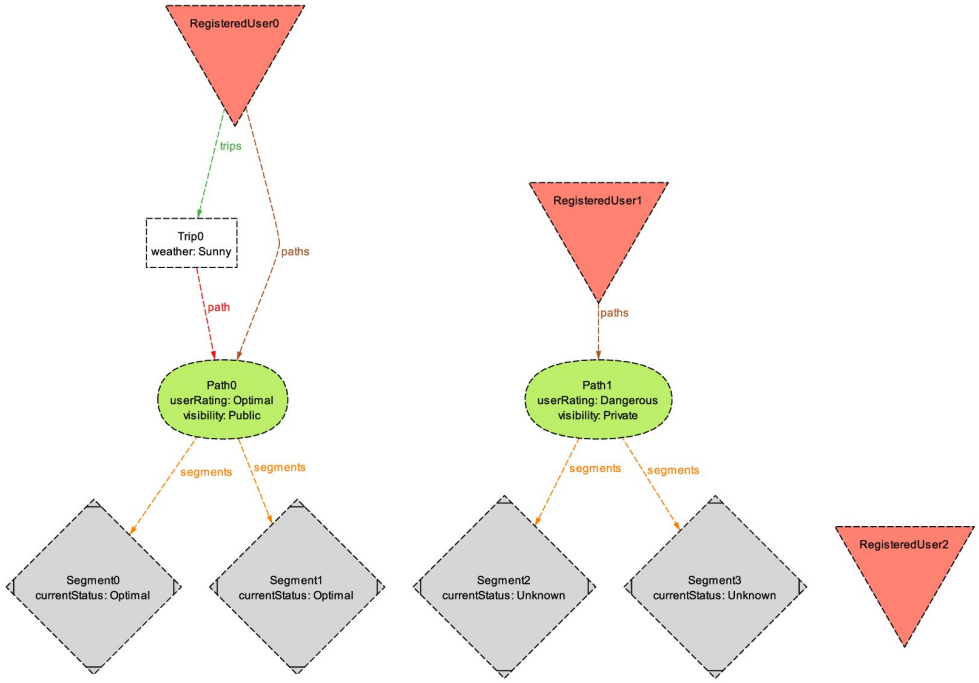


Figure 4.7 – Dynamic World, Step 6: User 3 Sign Up

The scenario continues in Figure 4.7 with the registration of a third User (User 3).

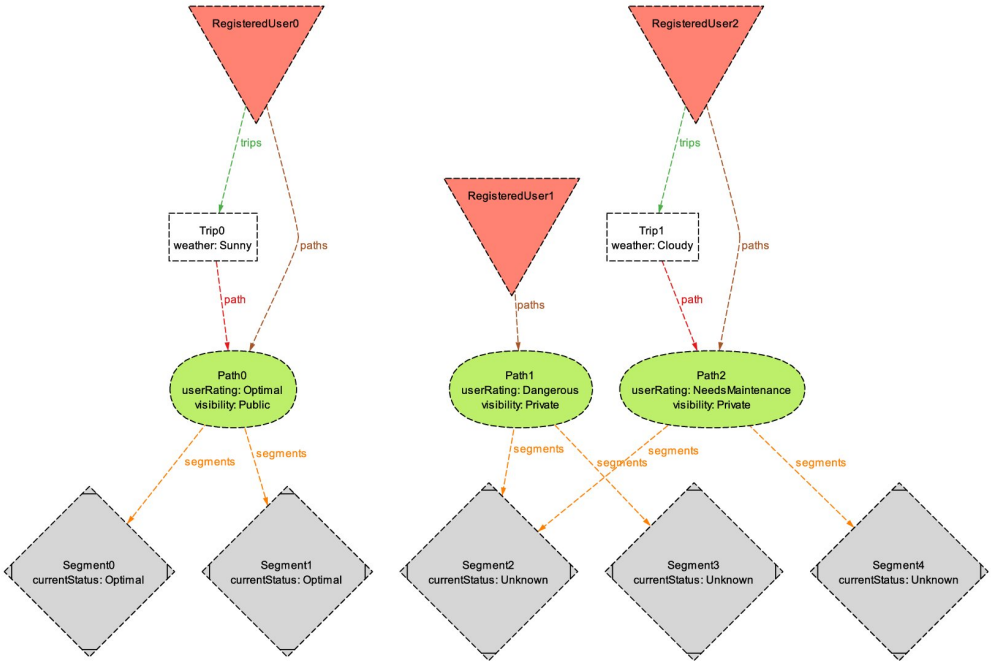


Figure 4.8 – Dynamic World, Step 7: User 3 records a Trip

Figure 4.8 highlights the Segment reuse logic. User 3 records a Trip that utilizes Segments already present in the System. Note that the private recording does not alter the status of these shared Segments, preserving the integrity of public data.

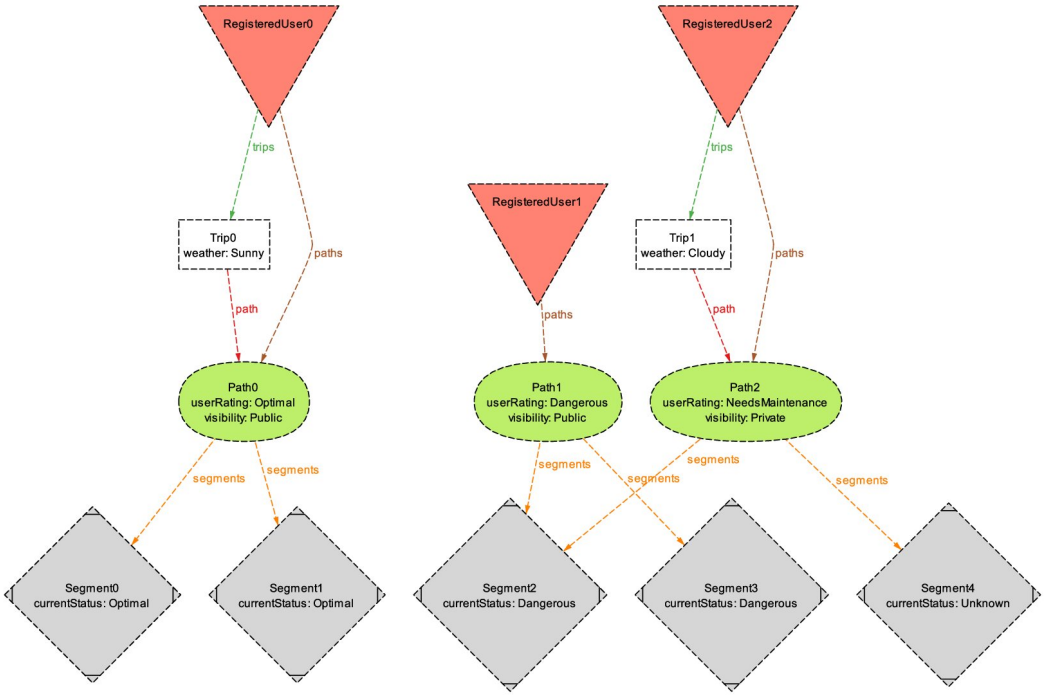


Figure 4.9 – Dynamic World, Step 8: Second Publication

Figure 4.9 shows the publication of User 1's Path, which updates the status of its specific Segments.

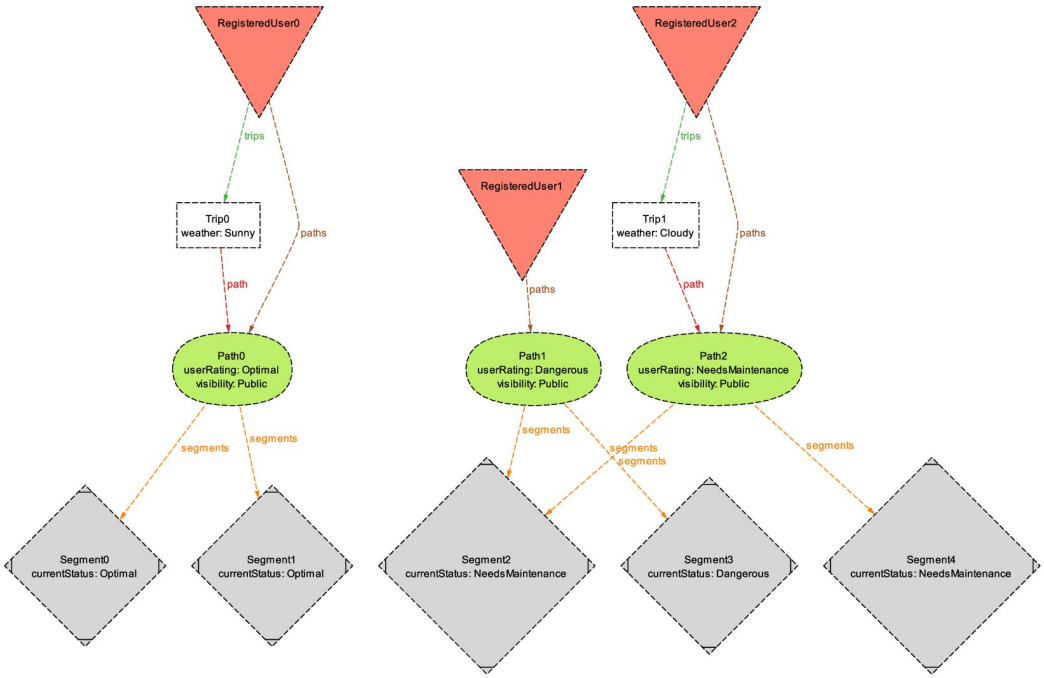


Figure 4.10 – Dynamic World, Step 9: Third Publication

Finally, Figure 4.10 shows the publication of User 2's Path, which updates the status of its specific Segments, overriding previous statuses, demonstrating that multiple publications can coexist without interference.

4.2.3 Assertion Checks

The formal properties of the Model were verified using the `check` command. The analyzer produced the following output:

```
No counterexample found. VisibilityIsolation may be valid.
No counterexample found. SegmentsStatusSync may be valid.
No counterexample found. OncePublicAlwaysPublic may be valid.
Counterexample found. SharingIsImpossible is invalid.
```

The results are fully consistent with the design requirements:

- The absence of counterexamples for the first three assertions confirms the **Safety**, **Liveness**, and **Possibility** properties (isolation of User data, correct status synchronization, and monotonicity of public visibility).
- The counterexample found for `SharingIsImpossible` is a desired result and serves as a **constructive proof**. By refuting the impossibility of sharing, the Model demonstrates that states where Paths share Segments are reachable and valid, confirming the correct implementation of the reuse logic.

Chapter 5

Effort Spent

Author	Section 1	Section 2	Section 3	Section 4
Daniele Toniolo	5	7	19	1
Stefano Ungaro	5	8	22	19
Matteo Zappa	5	7.5	22	1

Table 5.1 – Effort spent by each team member (unit is hours)

Chapter 6

References

6.1 Used Tools

- **Overleaf**[1]: LaTeX text editor for collaborative writing.
- **draw.io**[2]: creation of UML diagrams.
- **Alloy Analyser 6.2.0**[3]: formal analysis and model checking of the system.
- **Google Gemini 3 Pro**[4]: used for grammar, style, and coherence checks. Prompts included specific sections of this document with instructions to analyse and refine the text based on academic standards and logical consistency.

[1] https://www.overleaf.com
[2] https://www.draw.io
[3] https://www.alloytools.org
[4] https://gemini.google.com